基于覆盖搜索与滚动调度的无线电干扰源自主定位与清除

摘　要

针对问题一，建立**考虑 ±1° 测向误差的半平面交会定位模型**，利用定位区域的凸性将区域直径计算转化为最远顶点对搜索，并采用**旋转卡壳算法**在线性时间内求解。在此基础上，证明同直径覆盖圆的圆心只能为最远点对中点，给出逐顶点覆盖判据。构造三次测向形成的等边三角形反例，得到区域直径为 **20 m**、最小覆盖圆直径约 **23.09 m**，表明以区域直径为直径的圆**不一定能完全覆盖定位区域**。

针对问题二，结合首次示向度、接收距离及场地边界筛选备选测点，以源位置采样下的最大名义交会直径为指标，建立**基于交会区域直径的测点选址优化模型**。通过网格搜索与角域边界采样求解，将评价值不超过网格最优值 1.05 倍的测点组成**近优候选区域**。沿半径朝外、斜穿圆域与近切向斜穿三个算例的最优采样评价值分别为 **47.83**、**64.37** 和 **22.67 m**；固定所选测点加密源采样后，评价值最大变化约 **0.41%**。

针对问题三，面向数量未知的全向干扰源，建立**覆盖搜索与滚动调度模型**。以圆心和六个外圈点构造七点覆盖结构，其覆盖半径约为 992.689 m，小于 1000 m 的接收半径下限。融合历史测向、接收距离与无信号约束持续更新源位置可行域，通过 **粗细网格搜索**兼顾测后区域缩减与行动耗时。利用**最小包围圆与局部网格**分别制定单点清除和局部扫清方案，将搜索点与源区域纳入联合路线，并以最近邻和 2-opt 算法优化访问顺序。每轮仅执行首个任务，再根据新观测滚动规划，并通过测点复用减少重复移动。采用本问程序的正式测试累计清除 **28 个源**，清除率均为 **100%**，平均定位清除时间按源数加权后为 **225.84 s/源**。

针对问题四，面向全向源与 180° 定向源混合场景，构造 **25 点双环三角覆盖网**。通过三角剖分证明该点集满足任意发射朝向下的覆盖要求，最大剖分边长约 977.303 m，小于 1000 m 的接收半径下限。以最近成功测向点为参照开展**左右探测与自适应缩步**，双侧无信号时保留原位置可行域并将步长减半，避免因方向受限而错误排除可能源位置。结合最小包围圆判据、局部网格扫清与最小二乘估计点试探清除，动态选择清除方式；通过同点补测与搜索清除联合调度复用观测位置，在少源场景采用 **动态规划**优化访问顺序。三次正式测试累计清除 **35 个源**，清除率均为 **100%**，平均定位清除时间分别为 670.35、628.98 和 377.16 s，按源数加权后为 **532.88 s/源**。

关键字： 滚动调度　覆盖搜索　角域交会定位　旋转卡壳算法　网格搜索

一、 问题重述

1.1 问题背景

无线电频谱管理的关键任务之一是无线电干扰源的定位与清除，通常分为初步检测、区域性排查与精确定位三个基本阶段。前两个阶段通过固定监测站，移动监测车或无人机等设备采集并分析大范围信号以确定干扰源的分布区域。然而，由于环境遮挡、低功率干扰源等因素的影响，精准定位仍需技术人员携带测向机，结合信号场强变化和方向特征进行现场排查。为提高效率，某公司拟研发搭载于机器狗的自动定位清除系统，在危险环境中代替人工完成干扰源的定位与清除。

1.2 问题要求

本研究需结合题设条件，建立数学模型以系统研究以下四个问题：

1. 根据若干检测点坐标及测得的同一干扰源的示向度，通过交会定位法确定多边形区域，设计算法计算该区域任意两点间的最大距离并判断以该距离为直径的圆能否完全覆盖此区域。
2. 给定一全向干扰源在某检测点处的示向度，试给出第二个检测点的选择策略并确定有利于提高定位效果的候选区域。
3. 针对目标区域内数量未知的全向干扰源，派出一机器狗从原点出发，对干扰源进行定位与清除。在确保所有干扰源都能被清除的前提下，试设计机器狗的搜索定位与清除算法，尽量缩短任务完成时间。
4. 在问题（3）的基础上，进一步考虑全向与定向干扰源共存的情形，其中干扰源数量与定向干扰源的发射方向均未知。针对这一情况调整机器狗的搜索定位与清除算法，在确保全部干扰源被清除的前提下，尽量缩短任务完成时间。

其中，问题（3）与问题（4）均需通过模拟器开展演练及三次正式测试，验证所设计算法的有效性。

二、 问题分析

本题主要分为基于交会定位的区域直径计算与圆覆盖问题、以提高精度为目标的第二检测点选址优化问题，以及多干扰源的搜索定位与清除策略优化问题。

根据题目与附录信息，首先将示向度误差、信号接收范围等条件转化为干扰源位置的约束，以建立交会定位模型；其次，综合考虑包括机器狗移动、频道切换、检测与清除在内的各项时间成本，确定策略优化的核心目标；最后通过模拟器进行测试与演练，

根据成功清除比例与平均清除时间对策略进行评价，并完成正式测试。

2.1 问题一的分析

问题一要求我们根据检测点坐标及示向度，绘制多边形定位区域，给出计算该区域直径的算法，并探究以该直径为直径的圆能否覆盖定位区域。

根据题给信息，在考虑进 $\pm 1^\circ$ 的误差及示向度后，每个检测点可唯一确定一个包含干扰源的角域，多个检测点角域的交集即为定位区域。由于该区域具有凸性，其区域内任意两点之间距离的最大值必由一对最远顶点确定，考虑到穷举法遍历所有顶点对的时间复杂度较高，可利用定位区域的凸性，采用旋转卡壳算法进行优化计算。

对于圆覆盖问题，只由直径的长度大于等于区域内任意两点的距离不能直接推断同直径圆能够覆盖整个区域。因此，需以直径中点为圆心、区域直径的一半为半径，依次检验各顶点是否均位于圆内，并结合该最小包围圆的特点与反例分析给出结论。

2.2 问题二的分析

问题二要求我们根据已知的一个探测点关于某全向干扰源所测得的示向度，给出第二个探测点的选择策略及能得到较好定位效果的候选区域。问题的关键在于如何平衡信号接受条件与交会定位精度，使得第二次观测能够有效缩小干扰源位置的不确定范围。

首先，结合示向度误差、有效接收半径及目标区域边界，确定干扰源的可能位置范围，并分析各可能源位置下的接收条件，得到第二检测点的可行范围。随后，沿用问题一中定位区域直径的计算方法，以最坏情况下的区域直径作为选点评价指标。

求解时，对第二检测点的可行范围进行网格搜索，并对可能的源位置采样，以各采样情形下定位区域直径的最大值近似评价备选点。通过加密采样检验结果的稳定性，选出评价值最小的位置，再设置允许偏离最优值的比例，保留效果接近最优的备选点，形成候选区域。

2.3 问题三的分析

问题三要求在全向干扰源数量、频道及位置未知的前提下，保证机器狗能够清除全部干扰源并尽可能缩短完成任务的耗时。关键在于如何兼顾搜索的完整性、定位的可靠性与所选路径的效率。

由题，全向源具有有效接收半径不小于 1000 米的条件，可以利用这点将完整搜索转化为场地覆盖问题，通过记录检测点覆盖与频道扫描避免遗漏。另外，可以沿用问题一的角域交会方法，结合接收距离及是否有信号信息来更新源位置的可行域，并利用最小包围圆判断当前状态能否满足清除条件；对于较小区域，也可考虑进行局部多点清除。

在此基础上，可以将问题二的第二检测点选址算法扩展为综合考虑定位效果与行动耗时的决策。通过联合规划搜索点与源区域的访问顺序，重复利用检测位置，并根据最新观测实时规划行动路线，使搜索、定位与清除交织执行。最后，通过模拟器进行测试，

观察并记录清除比例与平均定位清除时间对策略进行评价。

2.4 问题四的分析

问题四在问题三的基础上引入了部分定向干扰源，其发射朝向未知。显然，此时即使检测点已经在干扰源的有效接收半径内，也有可能因为不在定向干扰源的覆盖方向上而接收不到信号。因此，本问的关键在于如何平衡距离与朝向问题，调整搜索覆盖、观测信息的利用及下一检测点的选择策略。

针对定向干扰源朝向未知的情况，我们可以重新对搜索点进行布局，利用三角剖分及凸包的性质，保证任意朝向的干扰源都能被成功检测。对于成功测向得到的角域及距离约束，我们依然可以沿用问题三的思路，但显然不能再利用无信号检测的结果来推断距离；取而代之的是从最近的成功检测点出发，向左右进行探索，并在双侧失败时缩短探索步长。[1]（AI 润色，2026-09-12）

最后，沿用最小包围圈与局部覆盖清除的方法，通过位置复用与动态路线规划尽可能减少移动耗时，并利用模拟器检验策略结果。

三、 模型假设

为简化问题，本文做出以下假设：

1. 将目标分布区域视为圆平面建立模型，忽略地形起伏。
2. 将干扰源、机械狗视为质点进行模型建立与分析。
3. 不对示向度误差作概率分布假设，一律按最坏情况（误差取到边界）处理。
4. 机械狗执行任务中，除机器狗移动时间、频道切换时间、检测时间、光学精确定位时间、清除时间外，其它事件消耗时间（如程序运行时间、机械狗调整姿态时间）忽略不计。

四、 符号说明

符号	说明	单位
Ω	目标圆域	–
G	干扰源位置	m
S_i	第 i 个检测点坐标	m
θ_i	第 i 次测向的示向度	°
α	测向误差界，取 1°	°
$W(S,\theta)$	测向误差角域	–
$P_k,\ P_{c,k}$	单源及频道 c 的定位可行域	–
$D(P)$	定位区域直径	m
$\widehat{J}(S_2)$	第二检测点评价值	m
$\widehat{\mathcal{C}}_\varepsilon$	允许偏离比例为 ε 的近优候选集	–
R_{cov}	搜索点集的场地覆盖半径	m
$O_c,\ r_c$	源可行域最小包围圆的圆心与半径	m
$\widehat{T}_c(q)$	候选测点 q 的时间评分	s
$k,\ h$	有效测向次数与多边形顶点数	–
R_c	频道 c 的源有效接收半径	m
$\mathcal{T}$	首次观测后的源位置可行域	–
X_t	当前机器狗位置	m
$\widehat{D}_c(q)$	候选测点的预测最大定位直径	m
$Q_\pm(s)$	缩放系数为 s 的左右探测点	m
$\widehat{G}_c$	源位置的最小二乘估计	m
ρ_{clear}	干扰源清除比例	–
T	任务总虚拟耗时	s
$\overline{T}$	平均定位清除时间	s/源

五、 问题一模型的建立与求解

5.1 基于测向误差角域的交会定位模型

针对问题一，本文将测向误差转化为关于干扰源位置的约束，通过多角域的交集构造定位区域，再利用凸多边形的几何性质计算其直径。这里的定位区域采用附录 2 所述的示向度角域交集，不额外叠加其他可能会引入其他边界的可行域模型。

设对同一干扰源共进行 k 次有效测向，第 i 个检测点为 $S_i=(x_i,y_i)$，示向度为 θ_i，且已知误差为 $\varepsilon=1°$。假定将检测点坐标作为已知输入，测向信息对应同一位置不变的干扰源，且真实方向与示向度的圆周角差不超过 ε。以上约束不要求误差相互独立，也

不要求其服从均匀分布。本文后续统一使用 k 表示检测点数，h 表示定位区域凸多边形顶点数，长度单位为米。

令待定位点为 $X=(x,y)$，定义

$$u_i=(\cos\theta_i,\sin\theta_i),\qquad d_i=X-S_i,\qquad t=\tan\varepsilon. \tag{1}$$

记二维叉积为 $a\times b=a_xb_y-a_yb_x$，则第 i 次测向所限定的角域为

$$W_i=\{X:u_i\times d_i-t(u_i\cdot d_i)\le 0,\quad -u_i\times d_i-t(u_i\cdot d_i)\le 0\}. \tag{2}$$

式(2)等价于 $|u_i\times d_i|\le t(u_i\cdot d_i)$。由于 $t>0$，两式相加即可推出 $u_i\cdot d_i\ge 0$，因此该表示已经排除了反向角域。

综合全部测向信息，定义

$$P_k=\bigcap_{i=1}^{k}W_i=\{X\in\mathbb{R}^2:AX\le b\}. \tag{3}$$

为所求的多边形定位区域。

由于每个角域均为两个闭半平面的交集，结合凸集的性质可知，P_k 是闭凸集。满足题设误差界的干扰源真实位置 G 属于每个 W_i，从而有 $G\in P_k$。区域非空且有界时，可定义并取到其直径

$$D(P_k)=\max_{X,Y\in P_k}\|X-Y\|_2. \tag{4}$$

与题中定义相同，该量描述了区域内所有相容位置间分离距离的最大值，即为所求直径的数学表示。

5.2 定位区域的判定与顶点构造

将各检测点的角域约束汇总后，定位区域可统一表示为

$$P_k=\left\{X\in\mathbb{R}^2\mid AX\le b\right\}.$$

显然，每一个不等式都代表一个半平面，定位区域就是这些半平面的交集。为了计算区域直径，我们应首先判断交集的状态，进而确定其边界顶点。

首先，检查交集是否为空，若交集为空集，则说明当前测向数据指向的定位区域无法对应同一个干扰源位置，此时无法构造有效定位区域。

若交集非空，还需判断其是否有界。直观上，若存在一个方向，使某个可行点沿该方向移动任意远后仍满足全部约束，则区域会沿该方向无限延伸。当非空区域存在这样无限延伸的非零方向时为无界区域，显然不具有有限直径；不存在时，区域有界，可以继续构造其顶点。

对于非空有界的交集，可采用边界交点枚举法确定定位区域。具体步骤如下：

1. **计算边界交点**。将每个半平面对应的边界写成直线方程，逐一计算两条非平行边界直线的交点。
2. **筛选可行交点**。将交点代入全部半平面约束，只有满足所有不等式的交点才予以保留。
3. **整理区域边界**。对可行点使用 Andrew 单调链算法求凸包，删除重复点和边上的冗余共线点，得到按逆时针顺序排列的顶点

$$V_0, V_1, \ldots, V_{h-1}.$$

以上方法能够完整确定定位区域：首先，半平面交集的每个顶点都是至少两条非平行约束边界的交会，因此会出现在枚举结果中；同时，只有满足全部约束的点会被保留下来，因此不会引入区域外的点。由于定位区域具有凸性，这些交点的凸包恰好就是原定位区域。

每次测向提供两条非冗余角域边界，因此，对于非退化的有界定位多边形，其顶点数满足

$$h \le 2k.$$

实际顶点数可能更少，因为部分测向约束不会形成最终区域的边界。

对于一些特殊情况：若区域退化为单点，其直径直接取 0；若退化为线段，则取两端点间距。

对于具有三个及以上顶点的凸多边形，将上述有序顶点序列输入旋转卡壳算法，继续求解区域直径。

5.3 区域直径的有限化与旋转卡壳求解

5.3.1 将区域直径转化为顶点间的最大距离

在上一小节中，我们得到了定位区域的有序顶点 $V_0, V_1, \ldots, V_{h-1}$。由上文可知，对于任意非空有界定位区域 P, 所求计算区域直径的数学表示为

$$D(P) = \max_{X,Y \in P} \|X - Y\|_2. \tag{5}$$

这里的 X 和 Y 代表定位区域内的任意候选位置，可以位于区域内部、边界或顶点。显然，直接搜索区域内的全部点不可行，因此需要利用凸性将搜索范围缩小到有限个顶点。

由于 P 是凸多边形，因此其中任意点 $X, Y \in P$ 都可以用各顶点的加权平均进行表示，即：

$$X = \sum_{i=0}^{h-1} \alpha_i V_i, \qquad Y = \sum_{j=0}^{h-1} \beta_j V_j, \tag{6}$$

其中 $\alpha_i, \beta_j \geq 0$，且 $\sum_i \alpha_i = \sum_j \beta_j = 1$。因此，

$$\begin{aligned} \|X - Y\|_2 &= \left\| \sum_{i,j} \alpha_i \beta_j (V_i - V_j) \right\|_2 \\ &\leq \sum_{i,j} \alpha_i \beta_j \|V_i - V_j\|_2 \\ &\leq \max_{i,j} \|V_i - V_j\|_2. \end{aligned} \tag{7}$$

也就是说，区域内任意两点之间距离的最大值就是最远顶点对之间的距离。由于顶点本身也是区域的一部分，故区域直径可表示为

$$D(P) = \max_{0 \leq i,j < h} \|V_i - V_j\|_2. \tag{8}$$

显然，只需在所有顶点对间寻找距离最远的顶点对，便可得到整个连续区域的直径。穷举全部 $\binom{h}{2}$ 个顶点对需要 $O(h^2)$ 时间，下面利用旋转卡壳进行优化，进一步减少搜索量。

5.3.2 平行支撑线与对踵点对

设一条与凸多边形接触的直线使整个多边形都位于直线的同一侧，这样的直线称为支撑线。当两条互相平行的支撑线夹住多边形时，两条线上的接触点构成**对踵点对**。旋转卡壳的基本思想是让这两条支撑线围绕多边形转动，并随转动更新接触顶点。

5.3.3 利用三角形面积推进支撑点

对逆时针排列的顶点，固定当前边 V_iV_{i+1}，定义

$$F_i(j) = (V_{i+1} - V_i) \times (V_j - V_i), \tag{9}$$

下标按模 h 循环计数。显然，$F_i(j)$ 是三角形 $V_iV_{i+1}V_j$ 面积的两倍，即

$$F_i(j) = \|V_{i+1} - V_i\|_2 \ \mathrm{dist}(V_j, L_i), \tag{10}$$

其中 L_i 是当前边所在的直线。由于 V_iV_{i+1} 的长度一定，面积最大的顶点也就是距离 L_i 最远的顶点，即对侧支撑线的接触点。

在计算过程中，逐次比较 $F_i(j+1)$ 与 $F_i(j)$：如果前者较大，就把对侧顶点从 V_j 修改为 V_{j+1}，直到面积不再增加。这里最大化的是点到当前边所在直线的距离，并不直接等于区域直径；仍需计算相应顶点对的欧氏距离。

随着凸多边形的边方向沿边界有序变化，对应的支撑位置也按边界顺序变化。因此，在当前边移到下一条边时，对侧支撑点可以从已有位置继续向前搜索，无须回到起点，这一特点极大地减少了每条边的遍历次数。

若 $F_i(j+1) = F_i(j)$ 且两者均达到最大值，则对侧支撑线同时经过两个相邻顶点，即与一条边重合。此时需要把两个对侧端点均纳入候选，完整处理平行边情形。

5.3.4 完整算法流程

综合以上描述，完整的算法流程如下：

1) 若区域为单点，返回 $D = 0$；若区域为线段，返回两端点距离。对 $h \geq 3$ 的凸多边形，初始化对侧指针 $j = 1$ 和最大距离平方 $M = 0$。
2) 依次处理各边 V_iV_{i+1}。当 $F_i(j+1) > F_i(j)$ 时推进 j，直至到达当前边的对侧支撑位置。
3) 分别计算 $\|V_i - V_j\|_2^2$ 和 $\|V_{i+1} - V_j\|_2^2$，与 M 比较并保存较大值及其端点。
4) 若对侧出现等面积的相邻顶点，则进一步检查 $\|V_i - V_{j+1}\|_2^2$ 和 $\|V_{i+1} - V_{j+1}\|_2^2$，更新最优结果。
5) 扫描全部边后，输出 $D = \sqrt{M}$ 以及对应最远点对。

5.4 计算结果与分析

由于本题未提供检测点数据，因此本文自行构建符合条件的算例进行验证。预设干扰源位置为 $G = (150, -250)$，根据检测点与 G 的位置关系生成示向度。G 仅用于生成验证数据，求解时只输入检测点坐标与示向度；具体输入见支撑材料中的算例文件。

表 **1**　典型布设下的定位区域直径

检测布设	检测点数 k	顶点数 h	直径 D/m
两检测点分别位于源的正东方和正北方	2	4	59.262511
于源的西南方向再增加一个检测点	3	6	59.262511
两检测点与源的夹角为 5°	2	4	1190.734431
两检测点均位于源的正东方且距离不同	2	—	无有限直径

表1中各有界算例的旋转卡壳结果均与对全部顶点进行穷举计算的结果一致。在 1000 个随机凸多边形的对照测试中，两种方法的最大数值差同样为 0，表明在此测试范围内直径的计算结果是可靠的。

不同布设的交会结果存在明显差异。新增第三站后，区域顶点数由 4 增至 6，但原最远点对仍被保留，因此直径未变。夹角较小的算例中直径显著增大，同方位算例则出现无界区域，说明定位具有不确定性且检测点的空间布设有关。

5.5 定位区域的同直径圆覆盖分析

给定前文求得的非空有界定位区域 P 的直径 D 及构成直径的最远顶点 A, B。以下讨论是否存在直径为 D 的圆覆盖整个区域。值得注意的是，当区域退化为单点时，半径为零的圆盘即可覆盖；下文仅考虑 $D > 0$ 的情形。

5.5.1 圆心确定与顶点覆盖判据

设某个半径为 $D/2$、圆心为 O 的圆盘能够覆盖 P，显然它同时包含了 A, B。

由于题目要求 O 在线段 AB 上，且到两端点的距离均为 $D/2$，即 A, B 是圆的直径，因此圆心只能取

$$O = \frac{A + B}{2}. \tag{11}$$

这说明，同直径圆能否覆盖区域并不是任意选取圆心的问题：若以 O 为圆心的圆盘不能覆盖，则移动圆心也无法在保持直径 D 的同时实现覆盖。

对区域顶点 $V_0, \ldots, V_{h-1}$，定义

$$\rho = \max_{0 \le i < h} \|V_i - C\|_2, \qquad \Delta = \rho - D/2. \tag{12}$$

显然，当任意到圆心的距离都小于等于半径的时候，同直径圆盘可以覆盖 P，即

$$\boxed{\text{存在同直径圆盘覆盖} P \quad \Longleftrightarrow \quad \rho \le D/2.} \tag{13}$$

也就是说，若 $\Delta > 0$，ρ 对应的顶点就是不能覆盖的直接证据。在已知直径端点的前提下，只需遍历全部顶点即可检验，检验复杂度为 $O(h)$。

5.5.2 由测向角域构造反例

为进一步判断是否能完全覆盖，构造三次测向形成等边三角形的反例。设干扰源真实位置为原点，检测点及示向度如表2所示，各测向误差均为 1°。

表 **2**　等边三角形反例的测向输入（坐标单位：米）

检测点	x_i	y_i	示向度/(°)
S_1	500	$500\sqrt{3} + 20/\sqrt{3}$	241
S_2	-1010	$-10/\sqrt{3}$	1
S_3	510	$-510\sqrt{3} + 20/\sqrt{3}$	121

通过交会定位法得到角域交集的三个顶点分别为

$$A = \left(-10, -\frac{10}{\sqrt{3}}\right), \quad B = \left(10, -\frac{10}{\sqrt{3}}\right), \quad T = \left(0, \frac{20}{\sqrt{3}}\right) \tag{14}$$

干扰源位于这三个顶点围成的多边形定位区域内，距各检测点约为 1000 米，选择 1200 米有效接收半径即可满足题设接收条件。

该三角形边长为 20 米，故 $D = 20$ 米。取底边 AB 为直径端点，候选圆的圆心为 $C = (0, -10/\sqrt{3})$、半径为 10 米，而

$$\|T - C\|_2 = 10\sqrt{3} \approx 17.3205 > 10. \tag{15}$$

显然点 T 超出了现有圆盘约 7.3205 米，如图1所示。结合圆心的唯一性，任意直径为 20 米的圆盘都无法完全覆盖该定位区域。

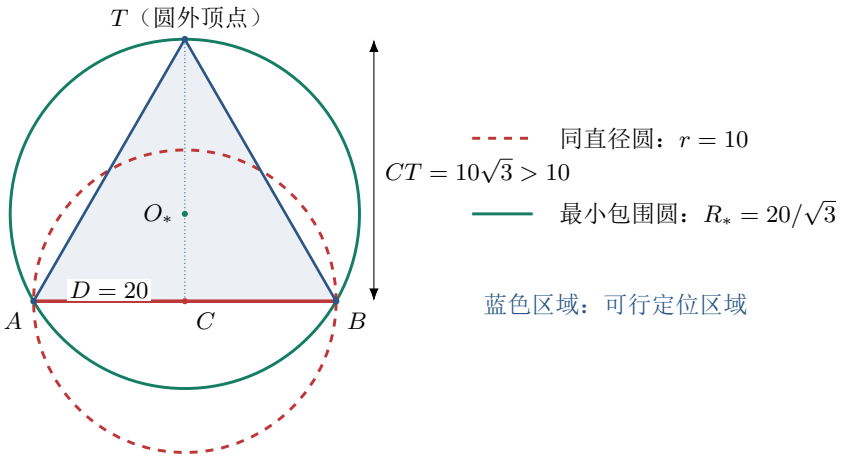


图 **1**　等边三角形反例：红色虚线圆盘不能覆盖顶点 T，绿色实线为最小包围圆。

5.5.3　最小包围圆解释与数值核验

记最小包围圆半径为 R_*。设计程序枚举候选圆并检查其是否包含全部顶点，得到表3中的结果。

表 **3**　区域直径与最小覆盖圆直径的比较（单位：米）

算例	D	$2R_*$	同直径圆能否覆盖
等边三角形反例	20.000000	23.094011	否
四边形反例	34.514803	34.532121	否
四边形对照例	49.385426	49.385426	是

通过观察与验证，以上反例具有共同的几何特征：其最小包围圆均由三个支撑顶点构成的锐角三角形的外接圆确定。其中，等边三角形由自身三个顶点支撑；四边形则由其中三个顶点支撑，剩余顶点位于圆内。

对照例则在数值容差下满足 $2R_* = D$，即能够被同直径圆完全覆盖。计算采用 10^{-7} 米的距离容差，两反例的直径差均明显大于该容差。

因此，以定位区域直径为直径的圆不一定能覆盖该区域，应结合顶点覆盖判据判断。

六、 问题二模型的建立与求解

6.1 首次观测下的干扰源可行域

针对问题二，本文首先根据第一次检测确定干扰源位置范围与第二次观测的接收条件，再以两个检测点角域交集的直径评价定位效果，通过网格搜索确定近似最优检测点及第二检测点的候选区域。

记首个检测点为 $S_1 = (x_1, y_1)$，测得的示向度为 θ_1。设目标圆域为 $\Omega = \{X : \|X\|_2 \leq 1800\}$，误差仍为 $\alpha = 1°$。角域的表示沿用问题一，将以 S 为顶点，θ 为中心方向的误差角域记为 $W(S, \theta)$。则首次检测可表示为：$W_1 = W(S_1, \theta_1)$。长度单位统一为米，距离均取欧氏距离。

由题，全向干扰源的最大可被有效接收距离为 $R \in [1000, 1500]$，且在距离不超过 5 m 时将返回近场信息而非示向度。既然我们已经得到了第一次检测的示向度，则干扰源位置 G 必须满足

$$\mathcal{T} = W_1 \cap \Omega \cap \{G : 5 < \|G - S_1\|_2 \leq 1500\}. \tag{16}$$

其中，目标圆域保证所有点都在场地内，角域反映测向的不确定性，距离上限由首次成功接收信号获得。数值采样以 5m 作为近端极限，由于其不返回示向度，故不将其视为有效的示向度观测。

令 $u(\varphi) = (\cos\varphi, \sin\varphi)$。对于符合要求的 S_1，从该点出发沿任意方向 φ 到达圆域边界距离的数学表达式为

$$t_\Omega(\varphi) = -S_1 \cdot u(\varphi) + \sqrt{[S_1 \cdot u(\varphi)]^2 + 1800^2 - \|S_1\|_2^2}. \tag{17}$$

因此，首次观测下的干扰源可行域关于角度与距离的参数化表示为

$$G = S_1 + r u(\varphi), \quad |\varphi - \theta_1|_{\text{circ}} \leq \alpha, \quad 5 < r \leq \min\{1500, t_\Omega(\varphi)\}. \tag{18}$$

6.2 第二检测点的接收约束

令 $r_1(G) = \|G - S_1\|_2$。首次检测成功接收说明 $R \geq r_1(G)$，因此，在给定可能源位置 G 时，与已有观测相容的最小接收半径为 $\max\{1000, r_1(G)\}$。能够保证第二次检测仍能接收到干扰源信号的理想可行集合为

$$\mathcal{A} = \{S_2 \in \Omega : \|S_2 - G\|_2 \leq \max\{1000, r_1(G)\}, \quad \forall G \in \mathcal{T}\}. \tag{19}$$

式 (19) 综合考虑了干扰源位置与接收半径的不确定性；任何首次检测距检测点超过 1000 m 的源，其接收半径最大值也必然超过 1000 m。

因此，只需约束各方向上距第一观测点不超过 1000 m 的源位置。

在干扰源可行域近端、远端左右两侧各取两个代表点，以每个点为圆心作圆相交得

$$\hat{\mathcal{A}} = \Omega \cap \bigcap_{j=1}^{4} \{S_2 : \|S_2 - V_j\|_2 \le 1000\}\,. \tag{20}$$

式 (20) 即为所求检测点近似可行域 $\mathcal{A}$。

6.3 定位效果的评价与选址模型

6.3.1 由接收可行性向精确定位转移

只对第二次检测点进行粗粒度筛查以确保能再次接收信号显然是不够的，显然这并不意味着定位区域也足够的小。不难发现，当两观测方向接近时，角域的交集往往十分狭长；在适当地向侧向移动后，交会区域狭长的几何形状能够得到改善，但实际效果还与观测距离有关。因此，本问沿用问题一的区域直径作为选点的核心指标。图2给出了干扰源可行域与检测点筛选的示例。[2]（AI 优化绘图，2026-09-11）

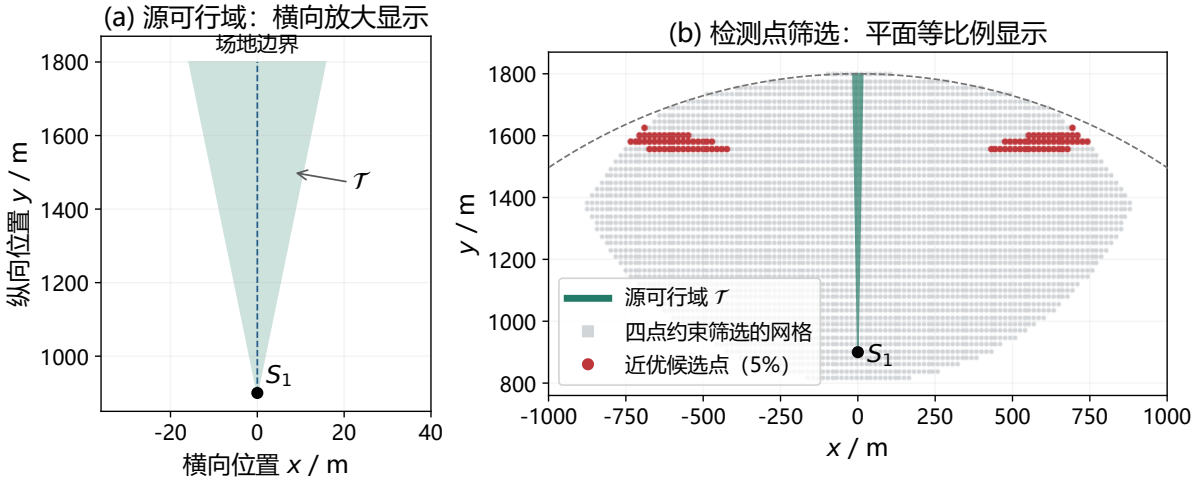


图 **2**　首次观测的源可行域与第二检测点筛选示意。左图横向放大以显示真实的 $\pm 1^\circ$ 角域；右图等比例显示四点约束筛选的网格及近优候选点。

6.3.2 源位置不确定时的模拟效果评价

在第二检测点的位置确定后，定位效果受干扰源位置的不确定性影响。因此，需要假设干扰源位于不同位置，分别计算固定的第二检测点与这些干扰源的交会区域大小并比较它们的表现。

首先，固定一个备选检测点 S_2，并在源可行域 $\mathcal{T}$ 中假设一个源位置 G 。根据此时的两个角域计算交会区域的直径并评估这一情形下的定位效果

$$D_0(G; S_2) = \operatorname{diam} P_0(G; S_2). \tag{21}$$

直径越小，说明模拟观测后得到干扰源的位置范围越集中。

随后，保持 S_2 不变，依次改变假设源位置 G。由于源的真实位置未知，采用其中最大的定位直径作为该检测点的评价值。数值计算时，取源采样点 $G_1,\dots,G_N$，得到

$$\widehat{J}(S_2)=\max_{1\le j\le N} D_0(G_j;S_2). \tag{22}$$

该值反映了在检查的所有源位置中，第二检测点放在 S_2 时最差的定位表现。对其他备选点进行相同计算，选择评价值最小的位置，即为近似最优的第二检测点。

6.4 网格搜索与近优候选区域的构造

6.4.1 可行位置与源位置的离散化

理想集合(19)包含连续多个源位置约束，因此实际使用的有限约束集合如式 (20)所示。对截断半径 c，定义

$$V_c=\bigcup_{s=\pm1}\left\{S_1+5u(\theta_1+s\alpha),\ S_1+\rho_s(c)u(\theta_1+s\alpha)\right\},\quad \rho_s(c)=\min\{c,t_\Omega(\theta_1+s\alpha)\}. \tag{23}$$

用 V_{1000} 中的四点检查 $\|S_2-V\|_2\le 1000$。距离函数的凸性保证四点检查能完全覆盖其凸包。

以 $u_1=u(\theta_1)$ 为纵向单位向量，以 $n_1=(-\sin\theta_1,\cos\theta_1)$ 为侧向单位向量，写成

$$S_2=S_1+au_1+bn_1,\qquad a\in[-1800,1005],\quad b\in[-1000,1000]. \tag{24}$$

分别将两个区间等分为 130 个网格坐标，删除场地外及不满足四点接收约束的位置，得到网格集合 $\widehat{\mathcal{A}}_h$。其中 a、b 分别表示前向与侧向位移，便于解释选点结果。

由于无法证明最大直径一定是在端点处取得，故误差评价不能仅检查四个端点。沿角域的两条边各取 100 个等距干扰源位置，组成 $\mathcal{G}_N$ ，计算

$$\widehat{J}_{\min}=\min_{S_2\in\widehat{\mathcal{A}}_h}\widehat{J}(S_2). \tag{25}$$

由于当前模型仅在采样评价有限的点之间进行比较，有限采样源可能会漏掉关键位置，网格也可能错过连续最优点，故 $\widehat{J}_{\min}$ 只代表当前离散模型的最优值。

6.4.2 候选点筛选与求解过程

综合以上描述，完整的求解过程如下：

1) 根据 S_1、θ_1 构造接收约束代表点与源采样点。
2) 枚举局部网格并检查接收与场地约束，得到备选检测位置。

3) 遍历源采样点，以最大名义交会直径评价备选位置，剔除产生无界交集的点。

4) 比较有限评价值，输出近似最优点及评价值，并按允许偏离比例筛选候选区域。

若备选网格点数为 M、源采样数为 N，则双站直径计算的单次规模固定，主要计算量为 $O(MN)$。取允许偏离比例 $\varepsilon \geq 0$，定义

$$\widehat{\mathcal{C}}_{\varepsilon} = \{S_2 \in \widehat{\mathcal{A}}_h : \widehat{J}(S_2) \leq (1+\varepsilon)\widehat{J}_{\min}\}. \tag{26}$$

候选面积以点数乘网格单元面积估计。ε 用于放宽选点要求。

6.5 算例结果与候选区域分析

由于问题二并未指定首次观测的坐标与示向度，因此采用三个场景进行比较分析：A 为 $S_1 = (0, 900)$、$\theta_1 = 90°$，B 为 $S_1 = (1200, 700)$、$\theta_1 = 120°$，C 为 $S_1 = (1500, 900)$、$\theta_1 = 120°$。三者分别对应沿半径朝外、斜穿圆域与近切向斜穿。所有结果均采用同一网格与干扰源采样密度。

表 **4**　三场景的近似最优检测点与 $\varepsilon = 5\%$ 候选规模

场景	有限评价点数	$\widehat{J}_{\min}$/m	最优 (x_2, y_2)/m	候选点数	面积/10^4 m^2
A	3740	47.83	(565.89, 1578.84)	94	3.17
B	2510	64.37	(307.83, 1144.51)	14	0.47
C	3215	22.67	(1123.87, 1039.86)	24	0.81

三场景通过接收约束筛选的点数分别为 3870、2689、3318，表4 仅统计其中具有有限评价值的点。A 场景关于首次观测射线对称，另一侧存在等价最优点，表中仅列一侧。

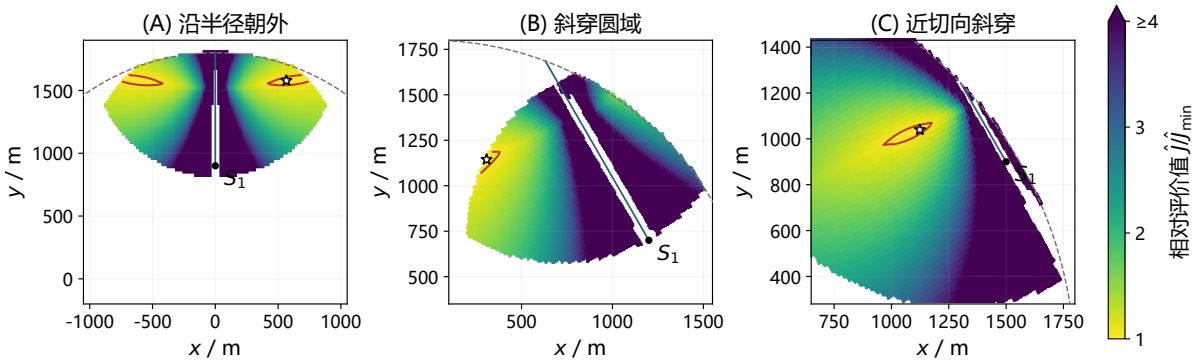


图 **3**　三场景的相对评价值与 $\varepsilon = 5\%$ 候选区域。红线表示候选边界，星号表示一处网格最优点；颜色统一表示 $\widehat{J}/\widehat{J}_{\min}$，大于等于 4 的值合并显示，空白处不参与有限值比较。

由图3可知，相同接收可行范围内的定位质量存在明显差异，通常较优的位置具有一定的侧向位移，能够改善首次方向难以确定的距离辨识问题；但前向位移与侧向位移

需要联合选择，不能仅由交会角确定。场景 A 的候选区呈两侧对称分布，B、C 的近优区域则在所示条件下偏向场地内部一侧。

取 $\varepsilon = 5\%$ 时，三场景候选点占有限评价点的比重分别为 2.51%、0.56% 、0.75%，评价阈值为 50.22m、67.59m、23.81 m。这说明接收可行范围仍需进一步筛选。场景 C 的源可行距离较短，取得较小评价值；但三场景同时改变了位置和边界条件，不能据此推出普遍单调关系，也不能断言所有最优点均贴近场地边界。

6.6 参数影响与数值结果的适用范围

6.6.1 允许偏离比例与候选规模

固定网格选取与源采样，分别对 $\varepsilon = 2\%, 5\%, 10\%, 20\%$ 进行筛选。由前文定义可知，ε 增大时候选区域单调扩张。图4同时展示候选面积与评价阈值，反映精度要求和选点灵活性之间的取舍。

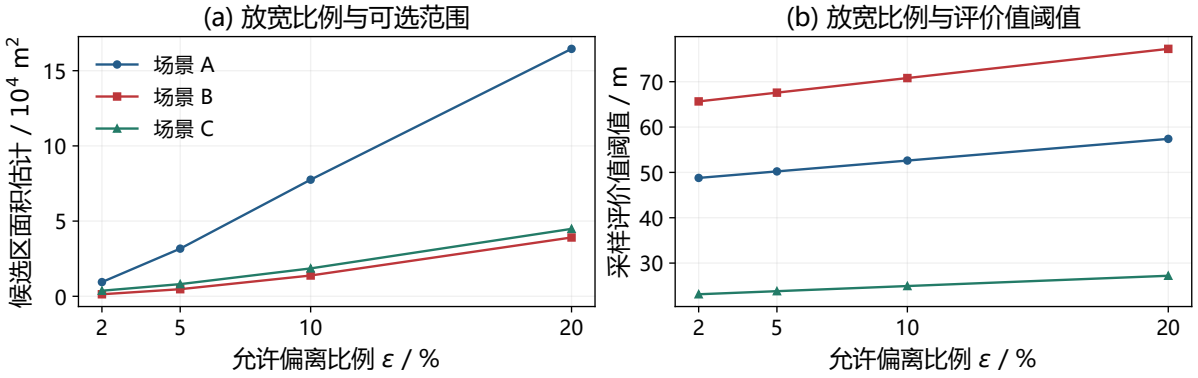


图 **4** 允许偏离比例对候选规模与采样评价阈值的影响。折线连接四个离散计算结果，不表示已拟合连续函数关系。

对与以上三个场景，ε 从 5% 增至 10% 时，面积估计的变化程度较大而评价阈值的变化并不明显，故可将 5% 作为本组算例的初始选择，再按实际情况进行调整。

6.6.2 源位置采样的局部核验

固定表4选出的检测点，分别采用每条角域边界 50、100、200 点进行计算，并将角域内部均分为 21 个角度、每个角度 200 个距离点进行比较，每个检测点的评价值如下表所示

表 **5** 固定所选检测点时的采样密度比较，单位：**m**

场景	每条边各 50 点	每条边各 100 点	每条边各 200 点	区域内 21 × 200 点
A	47.7954	47.8261	48.0236	48.0236
B	64.3667	64.3667	64.3667	64.3667
C	22.6727	22.6727	22.6727	22.6727

加密采样后，场景 A 的评价值增加约 0.41%，场景 B、C 基本不变，说明三个所选检测点的评价结果较为稳定，但仍需进一步检验整个候选区域。

综上，本节通过接收条件筛选和定位效果比较，得到第二检测点的近似最优候选区域。该结果基于有限采样与简化测向模型，属于数值近似；后续可进一步完善测角误差与接收条件的处理，并在问题三、四中考虑移动耗时和路径选择。

七、 问题三模型的建立与求解

问题二通过接收约束对第二检测点进行筛选，并将交会区域直径作为标准评价定位效果。本问延续这一思路并将其推广到多源问题；通过记录每次观测得到的可行域，在保证接收的候选点中，同时考虑移动耗时选择与可行域的缩减效果，并加入未知频道搜索、清除判定与多源访问顺序，形成搜索、定位、清除并行执行的控制策略。

7.1 任务状态与总时间目标

本问沿用前文定义的数学符号，记机器狗的当前位置为 X_t 、当前检测的频道为 b_t ，未知频道集合为 $\mathcal{U}_t$，已发现和已清除的频道分别表示为 $\mathcal{F}_t, \mathcal{C}_t$。对每个已发现源 c ，维护其位置可行多边形为 $P_{c,k}$，其中 k 为该频道已探查的有效示向度数。初始时刻 $X_0 = (0, 0)$、$b_0 = 1$、$\mathcal{U}_0 = \{1, \ldots, 20\}$。

查看附录的计时规则，设总路程为 L，切换频道、检测、清除失败及清除成功的次数依次为 n_b, n_m, n_f, n_s，则总虚拟时间为

$$T = \frac{L}{5} + n_b + 5n_m + 3n_f + 5n_s. \tag{27}$$

式(27)即为本节的核心优化目标。

7.2 基于全域覆盖判据的未知频道搜索

已知全向干扰源的有效接收半径至少为 1000 m，记 S_t 为已完成所有未知频道扫描的位置集合，其对场地的覆盖半径定义为

$$R_{\text{cov}}(\mathcal{S}_t) = \max_{G \in \Omega} \min_{S \in \mathcal{S}_t} \|G - S\|_2. \tag{28}$$

若 $R_{\text{cov}} \leq 1000$ m，说明当前所有扫描位置已经完全覆盖了整个区域，任何存在的源都应至少在一个扫描点被检测到。因此，所有尚未出现信号的频道此时都可判定为空；若已发现 16 个不同频道的源，根据题意，此时也已经探测了到所有未知源，可以结束未知频道搜索。

根据多次测试，以圆心和半径 $r_0 = 1140$ m 的六个外圈点组成后备覆盖结构：

$$S_0 = (0,0), \qquad S_j = r_0\big(\cos(\phi + (j-1)\pi/3), \sin(\phi + (j-1)\pi/3)\big), \quad j = 1,\dots,6. \tag{29}$$

该布设的覆盖半径约为 992.689 m，小于接收半径下限，因而满足全域覆盖条件。如图5所示，七个半径为 1000 m 的接收圆的并集包含整个场地；最不利位置出现在相邻外圈站之间的场地圆周上，图中标出了其中一个位置 G_*。

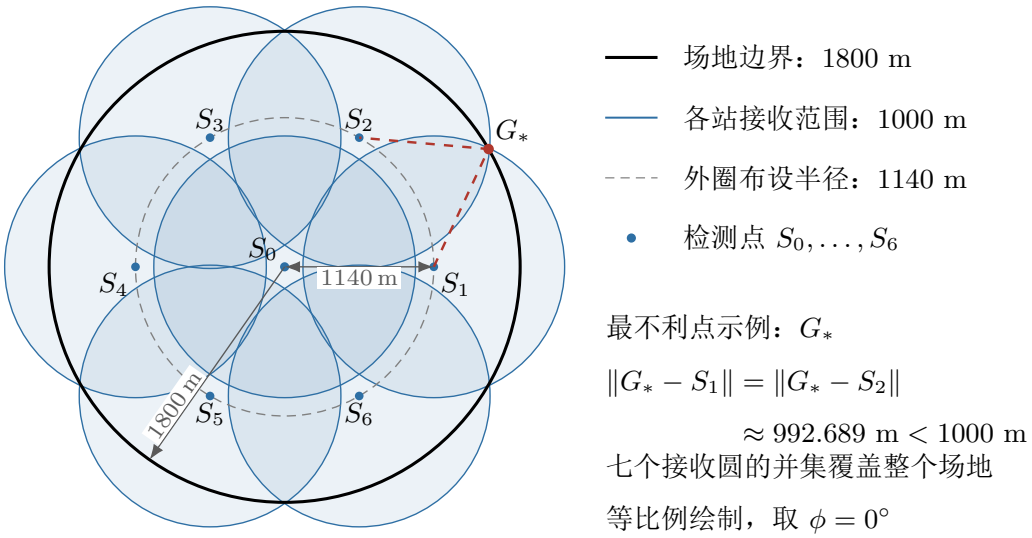


图 **5**　七个检测点的全域覆盖示意。黑色圆为目标场地边界，浅色区域为各站接收范围；红色虚线连接最不利点及其两个最近检测点。

定位与清除位置若进行了未知频道扫描，也加入 S_t。每轮枚举 $\phi = 0°, 5°, \dots, 55°$，并从相应的后备点中选取补点集合 $\mathcal{B}_t$，保证仍可满足 $R_{\mathrm{cov}}(\mathcal{S}_t \cup \mathcal{B}_t) \le 1000$ m 的条件。优先选择补点数最少的方案，再比较其与已知源区域联合规划的路线长度。

7.3　保留历史观测的源位置可行域

由题目可知，每次观测可以得到的信息有角域、场地及距离信息，本问需要保存全部历史观测信息。记 $\overline{\Omega}$ 和 $\overline{B}(S, 1500)$ 分别为场地圆盘与接收上界圆盘的外接正多边形，首次发现前取 $P_{c,0} = \overline{\Omega}$。每次 S 获得示向度 θ 令 $u_\theta = (\cos\theta, \sin\theta)$，通过一下方式对 P 进行更新：

$$P_{c,k+1} = P_{c,k} \cap W(S,\theta) \cap \overline{B}(S, 1500) \cap \{G : u_\theta \cdot (G - S) \ge 5\cos\alpha\}, \quad \alpha = 1°. \tag{30}$$

另外，无信号的探测记录也具有一定的利用价值。设一个未清除源在 S 点有信号、在 N 点无信号，则显然有 $\|G - S\| \le R_c < \|G - N\|$。平方展开后得到

$$2(N-S)\cdot G \le \|N\|_2^2 - \|S\|_2^2. \tag{31}$$

即未知源 G 总是位于 S 与 N 中更靠近 S 的一侧。此时作线段 SN 的垂直平分线即可将另一侧从可行域中剔除。因此，在发现源时可以调用此前保存的无信号记录；以后获得无信号记录时也可与已有有效测向点进行配对，避免白白浪费掉一次检测。

7.4 由问题二选址模型到基于时间的主动测向

接收约束与效果预测。本小节沿用问题二：接收可行性筛选、对可能源位置进行采样、保留近似最优点的核心框架，并针对当前多边形检查所有顶点。记 $P = P_{c,k}$、顶点集为 $V(P)$，主动测点集合为

$$\mathcal{A}_{c,k} = \{q : \max_{v\in V(P)} \|q - v\|_2 \le 1000\}. \tag{32}$$

由多边形的凸性，显然此时的测点 q 能够覆盖多边形区域内所有可能出现的信号源。

为了给出检测点的优化选择，下面通过对各备选检测点进行模拟预测并给出评估，从 P 取顶点、部分边中点及顶点均值组成样本集 G。设 $\beta(q,G)$ 为 q 指向 G 的方向角，$E = \{-1^\circ, 0, 1^\circ\}$ 为模拟误差，定义

$$\widehat{D}_c(q) = \max_{G\in\mathcal{G},\ e\in E} D\big(P \cap W(q, \beta(q,G) + e)\big). \tag{33}$$

上式将模拟角域与已有可行域 P 相交并计算测量后的区域直径 $\widehat{D}_c(q)$。该值越小，说明在所考察的情形下，该检测点越有利于缩小源的位置范围。若假设源位置距 q 不超过 5 米，则检测将返回近场信息，可直接清除，因此将该情形的后续定位需求记为零。

时间评分与粗细网格求解。设当前最小包围圈圆心为 O_c，采用以下方式对下次测向的选址进行评估

$$\widehat{T}_c(q) = \frac{\|X_t - q\|_2}{5} + 5 + w_k\frac{\|q - O_c\|_2}{5} + 0.85\frac{\widehat{D}_c(q)}{5}, \quad w_k = \begin{cases} 1.05, & k \le 2, \\ 0.38, & k \ge 3. \end{cases} \tag{34}$$

其中前两项为路程成本与检测成本，后两项为对测后接近源及继续定位代价的估计。早期观测时，较大的圆心距离权重会鼓励机器狗向源区域中心推进，以减少早期出现大跨度 V 形折返。

求解时，首先生成 125 m 粗网格，初步筛除不满足式(32)的点，并优先保持与历史测点至少 80 m 的基线，以避免重复获得相似信息。随后，结合检测距离与交会角进行几何初筛，同时保留靠近当前位置、源区域圆心及接近路径的候选点，避免遗漏移动成

本较低的位置。

对保留的候选点，计算预测区域直径 $\widehat{D}_c(q)$，并将其中的最小值记为 $\widehat{D}_{\min}$。沿用问题二保留近优检测点的思路，筛选满足

$$\widehat{D}_c(q) \le \eta_k \widehat{D}_{\min}$$

的位置。其中，已有有效测向次数 $k \le 2$ 时取 $\eta_k = 1.50$，否则取 1.18。该条件允许定位效果适度偏离本轮最佳预测值，为减少移动路程保留选择空间。再从这些位置中选取时间评分 $\widehat{T}_c(q)$ 最小的点，作为粗网格搜索结果。

最后，以所选粗网格点为中心，在附近按 30 m 步长建立细网格，重复接收条件筛选、定位效果预测与时间评分比较，确定下一检测点。由此，在问题二定位质量评价的基础上，将移动及后续处理成本纳入选点决策。

7.5 单点清除与局部扫清决策

由问题一的圆覆盖结论可知，不能将区域直径的一半视作覆盖半径。因此仍需对 $P_{c,k}$ 求最小包围圆：

$$r_c = \min_{O} \max_{v \in V(P_{c,k})} \|O - v\|_2, \qquad O_c \in \arg\min_{O} \max_{v} \|O - v\|_2. \tag{35}$$

若 $r_c \le 19\,\mathrm{m}$，则对于圆心 O_c，其到任一可能源的位置距离均小于 20 m，可以直接执行单点清除。之所以选择 19 m 而非 20 m 是为了给距离计算保存余量。

若接口区域已经较小但仍不满足上述条件，则可尝试局部扫清。对多边形取若干旋转方向的外接矩形，划分网格，使每个网格的半径长度不超过 18.5 m，于是单元内任一点到中心都严格小于 20 m。接下来只需按最短路线逐个尝试，成功即停止。若有 m 个点、到达首点的总路程为 L_g，则完成扫清最坏情况下的耗时为

$$T_g = L_g/5 + 3(m-1) + 5. \tag{36}$$

机器狗只有在预估局部扫清时间不超过主动测向时间加 45 秒时才会主动进行扫清，其中 45 秒为对主动测向后续定位成本的补偿。

7.6 多任务位置复用与滚动路线求解

前两小节给出了单个干扰源在考虑时间成本后的测点选择与清除方法。在确定多个干扰源且全域搜索尚未结束时，还需要确定各任务的执行顺序。因此，本节将统一对覆盖搜索与已知源处理两个任务进行规划，根据每步动作获得的信息动态调整下一步动作。

首先，将待补充的覆盖点与未清除源最小包围圆的圆心组成待访问节点，采用最近

邻法生成初始路线，再利用 2-opt 调整节点访问顺序以进一步缩短路程。若选中的下一访问节点是覆盖节点，则前往该节点扫描未知频道；若选中源节点，则根据第 **7.4**、**7.5** 节的方法，选择主动测向、单点清除或局部扫清。

在到达预定位置后，首先要检查该位置能否兼顾其他任务。在覆盖点完成未知频道扫描后，需要能够进一步缩小可行域的已知源补测；在定位区域进行清除时，若该点到最近已有扫描点的距离不超过 1200 米，则扫描未知频道并将该位置加入覆盖点集合。其次，若当前位置适合其他已知源测向，也可以进行有限次数的原地补测。虽然上述操作会产生额外时间消耗，但能够极大减少移动所需要的时间。

最后，每轮动作只执行路线中首个任务及位置复用操作，随后根据实时反馈重新规划新的路线。这样设计可以避免持续执行并不符合当前状态的访问顺序，减少额外路程开销。

上述算法的整体流程见图6。

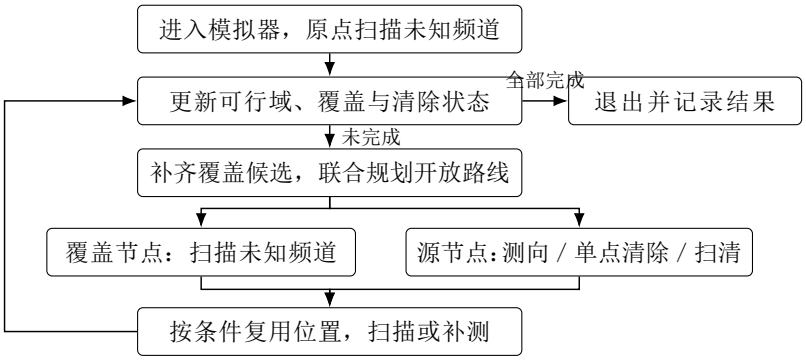


图 **6**　搜索、定位与清除的滚动决策流程。每轮处理一个节点，动作串行执行。

7.7　测试安排与结果记录

通过多次模拟演练检测覆盖搜索、可行域更新及多位置任务复用，再按照题目要求完成三次正式测试，由题，清除比例与平均定位清除时间分别为

$$\rho_{\text{clear}} = \frac{N_{\text{cleared}}}{N_{\text{true}}}, \qquad \overline{T} = \frac{T}{N_{\text{cleared}}}. \tag{37}$$

根据以上统计指标填写表6得

表 **6**　问题 **3** 正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
测试 1 案例编码	16	194.58	3
测试 2 案例编码	12	267.52	4
测试 3 案例编码*	11	566.62	7

* 测试 1、2 使用问题三程序；测试 3 因提交时程序选择失误，**误用了用于问题四的程序**。

正常提交结果的分析。测试 1、2 分别清除 16 个和 12 个干扰源，平均定位清除时间分别为 194.58 s 和 267.52 s，任务总虚拟时间分别约为 3113.28 s 和 3210.24 s。第二次

的平均时间比第一次高 37.49%，而总虚拟时间仅增加约 3.11%，说明平均时间的差异同时受到任务总耗时和已清除源数量的影响，不能仅凭该指标判断不同案例中的路线规划优劣。两次结果按已清除源数加权的平均时间为 225.84 s/源，该值仅用于描述这两次问题三程序的测试表现，不代替完整的三次正式提交记录。

第三次提交的版本差异。测试 3 记录的平均定位清除时间为 566.62 s，是由于该次提交误用了用于问题四的程序，与本节问题三程序的实验版本不一致，因此不能将三次结果作为同一算法的重复试验进行性能评价。本文保留该次正式结果，如实标注提交过程中的程序误用。

八、 问题四模型的建立与求解

在问题三中，我们已经建立了如何对全向源进行覆盖搜索、角域交会以及滚动调度的全套方法，本问在保留前问的任务状态及总虚拟时间目标的前提下，重点对依赖干扰源全向辐射作为前提条件的环节进行优化：以任意朝向下的覆盖代替距离覆盖，放弃由无信号推断源的距离，并以成功测向点为起点展开后续探究。

8.1 适应任意朝向的搜索网构造

记目标圆域为 Ω，定向源位置为 G，发射方向的单位向量为 $n_\varphi = (\cos\varphi, \sin\varphi)$。由于定向干扰源的有效覆盖范围是 180°，包含边界，则其方向条件可表示为如下闭半平面

$$H(G,\varphi) = \{S : n_\varphi^{\mathsf{T}}(S-G) \geq 0\}. \tag{38}$$

因此，本问搜索点集在满足问题三距离条件的同时，还需满足以上方向条件，即

$$\forall G \in \Omega,\ \forall \varphi \in [0, 2\pi), \quad \exists S \in \mathcal{S}: \quad \|S-G\|_2 < 1000, \quad S \in H(G,\varphi). \tag{39}$$

显然，假设一位于场地最东侧且向东发射的干扰源，显然仅靠问题三提出的图5的布设无法检测到该源。为此，本问设置了内外双圈搜索点，允许机器狗在目标区域外进行检测。

搜索点布局。取圆心 $O = (0,0)$、内环半径 $r = 950$ m，外环半径 $R_o = 1800/\cos 15^\circ$，构造

$$\begin{aligned} I_j &= r\big(\cos(30j^\circ), \sin(30j^\circ)\big), \\ E_j &= R_o\big(\cos((15+30j)^\circ), \sin((15+30j)^\circ)\big), \qquad j = 0, \ldots, 11. \end{aligned} \tag{40}$$

此时搜索点集 $\mathcal{S} = \{O, I_0, \ldots, I_{11}, E_0, \ldots, E_{11}\}$ 内共有 25 个点，内外环错开 15°，

其正十二边形的内切圆恰为场地圆，包含整个 Ω，具体布局及三角剖分见图7。

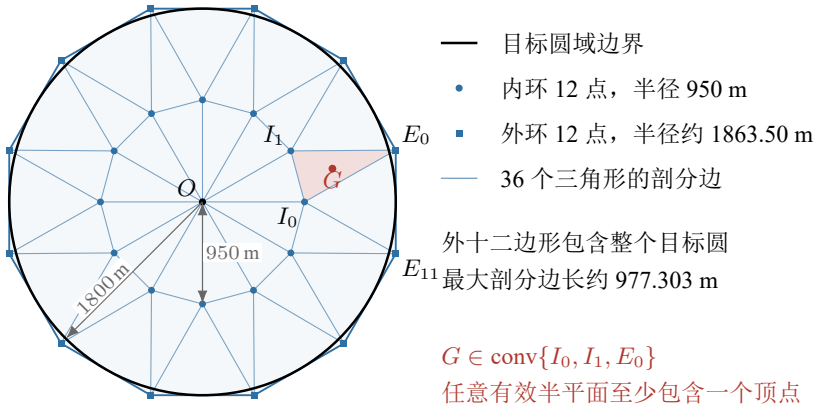


图 **7**　**25** 点搜索网及三角剖分。黑色圆为场地边界，蓝色点和方点分别为内、外环站；阴影三角形示意源与邻近搜索点的凸包关系，连线表示剖分边而非行走路线。

覆盖保证的证明。按图7对上述 25 个点进行连接，得到

$$\triangle O I_j I_{j+1}, \qquad \triangle I_j I_{j+1} E_j, \qquad \triangle E_{j-1} E_j I_j, \quad j = 0, \ldots, 11.$$

其中前 12 个三角形铺满内十二边形，后两类，即内外圈之间的 24 个三角形铺满剩余区域其边长只有以下四类：

$$\begin{aligned} &\ell_1 = r = 950, && \ell_2 = 2r \sin 15^\circ \approx 491.756, \\ &\ell_3 = 2R_o \sin 15^\circ \approx 964.617, && \ell_4 = \sqrt{r^2 + R_o^2 - 2rR_o \cos 15^\circ} \approx 977.303. \end{aligned} \tag{41}$$

因此最大剖分边长为 $\ell_{\max} \approx 977.303 < 1000$ m。说明对任意 $G \in \Omega$，任取一包含它的三角形，其到三角形任一顶点距离的最大值都不会超过 $\ell_{\max}$，即满足距离条件。

再考察任意发射方向 n_φ。若三个顶点均位于无效侧，根据式(38)，有 $n_\varphi^{\mathsf{T}}(A_i - G) < 0$。将三式按 λ_i 加权相加，得

$$0 = n_\varphi^{\mathsf{T}} \left(\sum_{i=1}^{3} \lambda_i A_i - G \right) < 0,$$

显然上式是矛盾的。因此三角形区域三个顶点至少有一个能满足方向条件，显然该点即为满足式(39) 针对该源的可行检测点。

频道搜索的逻辑与问题三是基本相同的，在原点及内外圈搜索站依次扫描尚未发现的频道。在完成 25 点搜索后，将始终未出现信号的频道设置为空；若已发现 16 个不同频道，则可提前终止频道搜索。与问题三不同的是，本问不采用动态补齐接收圆的覆盖，而是只保留固定搜索网，不使用未经方向覆盖验证的临时测点替换它。

8.2 观测约束与无信号反馈的修正

对观测成功并返回示向度的检测点，仍可按 7.3 节的办法，将本次角域与历史可行域、1500m 接收上界及近距排除平面相交。接收圆盘与场地仍使用正多边形进行近似，使满足误差界的真实源位置保留在可行域内。返回近距信息时，直接在当前点清除即可。

需要进行改进的是第 7.3 节提出的当得到有信号点 S 与无信号点 N 时的距离比较：第四问题设下显然不能推出未知源 G 总是离有信号点 S 更近的结论。因为无信号点的产生也可能是因为观测点恰好不在定向干扰源的有效覆盖范围内。因此本问在收到无信号时保持位置可行域不变，转而对探测测向与步长进行调整。

8.3 以成功观测为参照的左右探测

由于本问存在定向干扰源，因此采用问题三 7.4 节的候选点筛选仅能保证到所有可能源距离不超过 1000 米，这一条件在本题并不能据此保证获得新的示向度。为适应定向源的未知朝向，可以改用如下方法：

从最近一次成功测向位置 S 构造局部探测点，记示向度为 θ，令

$$u = (\cos\theta, \sin\theta), \qquad v = (-\sin\theta, \cos\theta), \qquad d_f = u^{\mathsf{T}}(O_c - S),$$

其中 O_c 为当前可行域最小包围圆的圆心；u 是测点指向源的估计方向。设置自适应步长及左右候选点

$$\begin{aligned} f_0 &= \min\{180, \max\{40, 0.55\max(d_f, 0)\}\}, \\ b_0 &= \min\{30, \max\{12, 0.18 f_0\}\}, \\ Q_{\pm}(s) &= S + s f_0 u \pm s b_0 v, \qquad s \in \{1, 1/2, \ldots, 1/64\}. \end{aligned} \tag{42}$$

这其中前向移动用于接近源，侧向横移用于形成新的交会区间。在源区域较近时可以缩短前进量 s，以减少候选探测点越过源进入无信号侧的次数。

每轮局部探测从 $s = 1$ 开始，按 Q_+、Q_- 的顺序依次尝试，当任一点成功时，根据该点更新可行域和参考位置，并对清除条件重新进行检查；若返回近距信息则直接清除。双侧均无信号时，显然此时的前进量 s 过大，令 $s \leftarrow s/2$，重新构造候选点。重复这一过程直到出现成功测向点。

8.4 清除判据与局部动作的调整

由于定向源光学清除不受方向影响，因此 7.5 小节最小包围圆半径 $r_c \le 19$ m 时的单点清除结论保持不变。对细长可行域，改以该域直径两端点为主轴构造包围矩形，将两边等分，使得每个单元边长不超过 26 m。此时任一点到所在单元中心的距离满足

$$\frac{1}{2}\sqrt{d_u^2+d_v^2} \le \frac{26}{\sqrt{2}} \approx 18.385 < 20\ \text{m}. \tag{43}$$

因此，以最短遍历路线逐个遍历网格中心即可覆盖此细长可行域。

当定位范围虽然较小，却无法保证一次清除成功时，可以用最小二乘估计给出干扰源最可能出现的位置，即

$$\widehat{G}_c = \arg\min_{X} \sum_{i} \left[n_i^{\mathsf{T}} (X - S_i) \right]^2. \tag{44}$$

只有当 $r_c \le 70$ m 且 $\|\widehat{G}_c - O_c\| \le \max\{80, 1.8 r_c\}$，即当前最小包围圆不太大且估计点不离包围圆心太远时，机器狗才会前往该点尝试清除。设计此机制的目的是通过一次可能成功的清除尝试，争取节省一些测向与移动时间。即使试探失败，当前可行域也会被保留下来。

8.5 搜索清除联合调度与少源路线优化

沿用 7.6 节的位置复用与滚动规划思想，并可将执行过程分为两个阶段。阶段 A 对未知频道进行 25 点的覆盖搜索，在搜索站补测已知源，并加入可单点清除、可网格扫清的干扰源重新规划剩余路线；需专门定位的源仍依靠顺测积累信息。阶段 B 则是在搜索结束后，对尚未清除的源进行专门探测与清除。

同点补测。为尽量减少机器狗的移动距离，可以通过额外观测尽可能缩小可行域的大小。以当前最小包围圆的圆心为预测点，计算假设测向成功后的半径 $\widehat{r}_c(q)$，用 $g_c(q) = 1 - \widehat{r}_c(q)/r_c$ 来衡量缩小幅度。只有在半径缩小至少 10% 时予以测量，预计可直接达到 19 m 时优先保留。每个搜索站最多补测 4 个频道。

混合路线。与问题三相同，对已经确定的搜索站、单点清除及网格清除任务，采用多起点最近邻与 2-opt 生成动态最短路线；对尚需定位的源，用式(44)给出的预测位置辅助评分：

$$J(\pi) = L(\pi) + 0.12 \sum_{c} \min\{700, d(\widehat{G}_c, \pi)\}, \tag{45}$$

其中 $d(\widehat{G}_c, \pi)$ 为从当前位置到预测点路线的最短距离。考虑到预测位置的不准确性，设定该试图靠近预测源的绕路路线不能多走超过 120 m，否则将退回原线路。

少源阶段的进一步优化。阶段 B 以各源的清除圆心、网格首点或探测候选点作为路线代表点。当确认总源数为 10 至 13 时，每次停靠最多顺测另外 2 个高收益频道，以避免在 15 m 内重复尝试同一频道。采用动态规划优化访问顺序, 其余场景沿用常规策略。

8.6 算法流程与有效性检验

图8给出了对以上两阶段控制及单源反馈流程的总结。

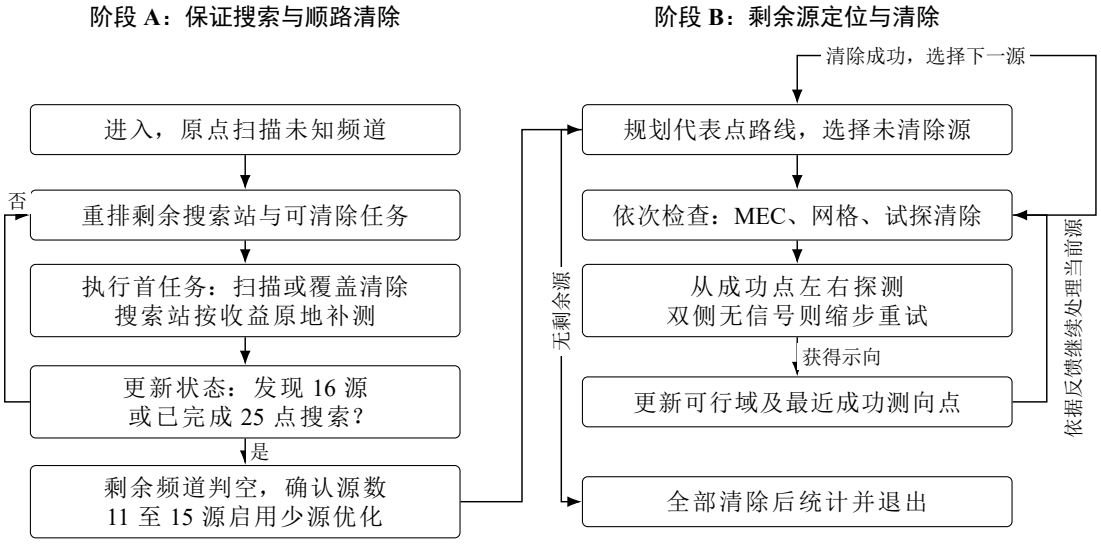


图 **8**　问题四核心流程。左侧为搜索与顺路清除，右侧为剩余源处理；无信号仅改变探测安排，不直接裁剪位置可行域。

按照题目要求完成三次正式测试，沿用式(37)计算清除比例与平均定位清除时间，结果见表7。

表 **7**　问题 **4** 正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间/s	程序运行时间/s
测试 1	10	670.35	4
测试 2	10	628.98	2
测试 3	15	377.16	3

清除效果。三次测试分别清除 10、10 和 15 个干扰源，均与对应案例的干扰源总数一致，清除比例均为 100%。结果表明，在这三个全向与定向源混合案例中，所设计的搜索、定位及清除策略均完成了全部目标的处理，验证了算法在所测场景下的有效性。

定位清除时间。三次测试的总虚拟时间分别为 6703.577、6289.868 和 5657.457 s。测试 1、2 的源数量相同，但测试 2 的总耗时比测试 1 减少 6.17%，说明源数量不是决定任务耗时的唯一因素，源的位置、接收范围及定向覆盖条件也会影响搜索与定位过程。测试 3 虽然需要清除 15 个源，其总耗时却比测试 2 减少 10.05%，平均定位清除时间降低 40.04%。这一差异既包含总耗时的减少，也受到清除数量增加的影响，不能仅凭平均时间下降便认定某项优化取得了同等幅度的收益。

运行效率与综合评价。三次测试累计清除 35 个干扰源，按清除数量加权的平均定位清除时间为 532.88 s/源。因此，后续改进应重点关注机器狗的移动、检测及无信号探测等虚拟动作成本。上述结果支持策略在已测试案例中的可行性，但不同案例条件并不

相同，各优化环节的独立节时效果仍需结合动作日志或对照测试进一步判断。

九、 模型的评价与推广

9.1 模型的优点

1. **模型具有较强的理论性与可解释性。**本文将测向误差转化为半平面约束，通过多个角域交集刻画干扰源位置可行域，并利用凸多边形性质、旋转卡壳算法和最小包围圆完成区域直径计算与清除判定。各步骤均具有明确的几何意义，避免了仅以区域直径衡量覆盖范围可能产生的误判，提高了定位结果的可靠性。
2. **模型形成了较完整的动态决策体系。**模型并未将搜索、定位与清除相互割裂，而是以干扰源可行域为核心，根据实时观测不断更新状态，并综合考虑移动、测向和后续处理成本选择下一检测点。同时将覆盖节点与已发现干扰源统一进行路线规划，每次执行后重新决策，从而提高检测位置的复用率并减少不必要的移动路程。
3. **对全向与定向干扰源均具有较好的适应性。**针对全向源，模型利用最小接收距离构造覆盖搜索结构，保证未知干扰源能够被发现；针对定向源，则构造内外双圈搜索网，并借助三角剖分和凸组合性质保证不同发射方向下仍存在有效检测点。同时取消不再适用的无信号距离推断，使模型能够适应全向、定向干扰源共存的复杂情况。

9.2 模型的不足

1. **部分优化结果仍依赖离散化近似。**第二检测点选址以及主动测向位置的确定主要采用网格搜索和有限源位置采样，因此得到的是当前离散精度下的近似最优解，而非连续空间中的严格全局最优。虽然通过加密源位置采样验证了部分算例结果的稳定性，但仍可能存在网格未覆盖的更优检测位置或未被采样捕获的最不利情况。
2. **部分参数的选取具有一定经验性。**模型中的粗细网格步长、近优点筛选比例、主动测向评分权重以及局部扫清阈值等参数，主要依据几何特征和模拟实验进行设定。该方法在当前测试条件下具有较好的计算效率，但当场地规模、机器狗速度、测向精度或干扰源分布发生明显变化时，原有参数未必仍能保持较优性能。
3. **模型对真实无线电传播环境进行了简化。**本文主要依据题目给出的测向误差、接收半径和定向源覆盖角度建立模型，将接收范围近似为规则几何区域。实际环境中可能存在建筑遮挡、多径传播、信号衰减和地形限制等因素，使有效接收区域偏离理想模型。因此，当前方法更适用于题设条件明确的场景，实际应用

时仍需结合环境数据进行修正。

9.3 模型的推广与应用

本文模型可进一步推广至复杂场地和多智能体协同定位场景。对于存在障碍物或不规则边界的区域，可将目标区域划分为若干子区域，并以实际可行路径替代欧氏距离完成覆盖与路线规划；在大范围搜索任务中，可采用多台机器狗或无人机协同完成不同区域及不同干扰源的定位。同时，还可将固定的测向误差角域推广为概率分布或置信区域，并结合贝叶斯更新、鲁棒优化或信息增益评价动态选择检测位置，从而提高模型对测量噪声、传播环境变化及复杂实际场景的适应能力。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于语言润色、代码调试、辅助绘图，详细使用情况见支撑材料。

参考文献

[1] Jakkala K, Akella S. Fully differentiable sensor placement and informative path planning[J/OL]. The International Journal of Robotics Research, 2026, 45(8): 1247–1284. https://doi.org/10.1177/02783649251384993.

[2] Moon B, Suvarna N, Jong A, et al. IA-TIGRIS: an incremental and adaptive sampling-based planner for online informative path planning[J/OL]. IEEE Transactions on Robotics, 2026, 42: 1695–1713. https://doi.org/10.1109/TRO.2026.3672542.

[3] Toussaint G T. Solving geometric problems with the rotating calipers[C/OL]//IEEE MELECON '83, 1983: 1. https://www-cgrl.cs.mcgill.ca/~godfried/publications/calipers.pdf.

[4] Elzinga D J, Hearn D W. The minimum covering sphere problem[J/OL]. Management Science, 1972, 19(1): 96–104. https://doi.org/10.1287/mnsc.19.1.96.

[5] Bishop A N, Fidan B, Anderson B D O, et al. Optimality analysis of sensor-target localization geometries[J/OL]. Automatica, 2010, 46(3): 479–492. https://doi.org/10.1016/j.automatica.2009.12.003.

[6] Dogancay K. Optimal geometries for AOA localization in the Bayesian sense[J/OL]. Sensors, 2022, 22(24): 9802. https://doi.org/10.3390/s22249802.

[7] Bánhelyi B, Palatinus E, Lévai B L. Optimal circle covering problems and their applications[J/OL]. Central European Journal of Operations Research, 2015, 23(4): 815–832. https://doi.org/10.1007/s10100-014-0362-7.

[8] Vander Hook J, Tokekar P, Isler V. Algorithms for cooperative active localization of static targets with mobile bearing sensors under communication constraints[J/OL]. IEEE Transactions on Robotics, 2015, 31(4): 864–876. https://doi.org/10.1109/TRO.2015.2432612.

附录

A　问题一求解代码

[3]（AI 调试，2026-09-11）

```
from __future__ import annotations
import math
from itertools import combinations
import rotating_calipers as rc
__all__ = [
    "wedge_halfplanes",
    "normals_positively_span",
    "intersect_halfplanes",
    "region_from_bearings",
    "disk_polygon",
    "clip_halfplane",
]

def wedge_halfplanes(sx: float, sy: float, theta_deg: float, alpha_deg: float = 1.0):
    th = math.radians(theta_deg)
    al = math.radians(alpha_deg)
    ux, uy = math.cos(th), math.sin(th)
    t = math.tan(al)

    cuS = ux * sy - uy * sx          # cross(u, S)
    duS = ux * sx + uy * sy          # dot(u, S)
    return [
        (-uy - t * ux, ux - t * uy, cuS - t * duS),
        (uy - t * ux, -ux - t * uy, -cuS - t * duS),
        (-ux, -uy, -duS),
    ]

def normals_positively_span(hps) -> bool:
    angs = sorted(math.atan2(b, a) for a, b, _ in hps)
    n = len(angs)
    if n < 2:
        return False
    gaps = [angs[i + 1] - angs[i] for i in range(n - 1)]
    gaps.append(angs[0] + 2 * math.pi - angs[-1])
    return max(gaps) < math.pi - 1e-12

def _line_intersect(l1, l2):
    a1, b1, c1 = l1
    a2, b2, c2 = l2
    det = a1 * b2 - a2 * b1
    if abs(det) < 1e-14:
```

```
        return None
    return ((c1 * b2 - c2 * b1) / det, (a1 * c2 - a2 * c1) / det)

def intersect_halfplanes(hps, scale: float = 1.0):
    if not hps:
        return "unbounded", []
    if not normals_positively_span(hps):
        return "unbounded", []
    tol = 1e-9 * max(1.0, scale)
    cand = []
    for l1, l2 in combinations(hps, 2):
        p = _line_intersect(l1, l2)
        if p is None:
            continue
        if all(a * p[0] + b * p[1] - c <= tol for a, b, c in hps):
            cand.append(p)
    if not cand:
        return "empty", []
    poly = rc.simplify_polygon(rc.convex_hull(cand))
    return ("ok", poly) if poly else ("empty", [])


def disk_polygon(cx: float, cy: float, radius: float, n_sides: int = 720):
    """圆盘的内接正 n 边形顶点（逆时针）。"""
    return [
        (cx + radius * math.cos(2 * math.pi * i / n_sides),
         cy + radius * math.sin(2 * math.pi * i / n_sides))
        for i in range(n_sides)
    ]

def clip_halfplane(poly, hp, tol: float = 1e-9):
    """用单个半平面 a.x + b.y <= c 裁剪多边形（Sutherland-Hodgman）。"""
    a, b, c = hp
    n = len(poly)
    if n == 0:
        return []
    out = []
    for i in range(n):
        p = poly[i]
        q = poly[(i + 1) % n]
        dp = a * p[0] + b * p[1] - c
        dq = a * q[0] + b * q[1] - c
        pin, qin = dp <= tol, dq <= tol
        if pin:
            out.append(p)
        if pin != qin:
            t = dp / (dp - dq)
            out.append((p[0] + t * (q[0] - p[0]), p[1] + t * (q[1] - p[1])))
```

```
    return out

def region_from_bearings(
    detections,
    alpha_deg: float = 1.0,
    region_disk_radius=None,
    reception_max=None,
    n_sides: int = 720,
):
    notes = []
    scale = 1.0
    for x, y, _ in detections:
        scale = max(scale, abs(x), abs(y))

    wedge_hps = []
    for x, y, th in detections:
        wedge_hps.extend(wedge_halfplanes(x, y, th, alpha_deg))

    if not region_disk_radius and not reception_max:
        status, poly = intersect_halfplanes(wedge_hps, scale)
        if status == "unbounded":
            notes.append(
                "角楔交集无界：各示向度方向锥存在公共方向，没有有限的定位区域直径。"
                "需增加一个不同方位的检测点，或叠加圆域/接收距离约束。"
            )
        return status, poly, notes
    seed = None
    if region_disk_radius:
        seed = disk_polygon(0.0, 0.0, region_disk_radius, n_sides)
        notes.append(
            f"叠加圆域约束 r <= {region_disk_radius:g} m"
            f"（内接 {n_sides} 边形，径向亏空 "
            f"{region_disk_radius * (1 - math.cos(math.pi / n_sides)):.4f} m）"
        )
    if reception_max:
        for x, y, _ in detections:
            disk = disk_polygon(x, y, reception_max, n_sides)
            seed = disk if seed is None else _clip_by_convex(seed, disk, scale)
            if not seed:
                break
        notes.append(f"叠加接收距离约束 <= {reception_max:g} m")
    if not seed:
        return "empty", [], notes
    tol = 1e-9 * max(1.0, scale)
    poly = seed
    for hp in wedge_hps:
        poly = clip_halfplane(poly, hp, tol)
        if not poly:
```

```
            break
    if not poly:
        return "empty", [], notes
    poly = rc.simplify_polygon(rc.convex_hull(poly))
    return ("ok", poly, notes) if poly else ("empty", [], notes)

def _clip_by_convex(subject, clipper, scale: float = 1.0):
    tol = 1e-9 * max(1.0, scale)
    clip = rc.rotate_to_ccw(list(clipper))
    m = len(clip)
    if m < 3:
        return []
    poly = list(subject)
    for i in range(m):
        p, q = clip[i], clip[(i + 1) % m]
        ex, ey = q[0] - p[0], q[1] - p[1]
        a, b = ey, -ex
        c = a * p[0] + b * p[1]
        poly = clip_halfplane(poly, (a, b, c), tol)
        if not poly:
            break
    return poly
```

```
from __future__ import annotations
import math
__all__ = [
    "cross",
    "convex_hull",
    "simplify_polygon",
    "polygon_area",
    "rotate_to_ccw",
    "rotating_calipers_diameter",
    "antipodal_pairs",
    "brute_force_diameter",
]
def cross(ox: float, oy: float, ax: float, ay: float, bx: float, by: float) -> float:
    return (ax - ox) * (by - oy) - (ay - oy) * (bx - ox)

def dist(p, q) -> float:
    return math.hypot(q[0] - p[0], q[1] - p[1])

def _scale_of(poly) -> float:
    m = 1.0
    for x, y in poly:
        m = max(m, abs(x), abs(y))
    return m

```

```
def rotate_to_ccw(poly):
    s = 0.0
    n = len(poly)
    for i in range(n):
        x1, y1 = poly[i]
        x2, y2 = poly[(i + 1) % n]
        s += x1 * y2 - x2 * y1
    return list(poly) if s >= 0 else list(reversed(poly))

def convex_hull(points):
    pts = sorted({(float(x), float(y)) for x, y in points})
    if len(pts) <= 2:
        return pts
    def half(seq):
        out = []
        for p in seq:
            while len(out) >= 2 and cross(
                out[-2][0], out[-2][1], out[-1][0], out[-1][1], p[0], p[1]
            ) <= 0:
                out.pop()
            out.append(p)
        return out
    lower = half(pts)
    upper = half(reversed(pts))
    return lower[:-1] + upper[:-1]

def simplify_polygon(poly, rtol: float = 1e-9):
    pts = [(float(x), float(y)) for x, y in poly]
    n = len(pts)
    if n < 3:
        return pts
    eps = rtol * _scale_of(pts)
    keep = []
    for p in pts:
        if keep and dist(p, keep[-1]) <= eps:
            continue
        keep.append(p)
    if len(keep) >= 2 and dist(keep[0], keep[-1]) <= eps:
        keep.pop()
    pts = keep
    if len(pts) < 3:
        return pts
    for _ in range(len(pts)):
        m = len(pts)
        out = []
        for i in range(m):
            a, b, c = pts[(i - 1) % m], pts[i], pts[(i + 1) % m]
            lac = dist(a, c)
```

```
            if dist(a, b) <= eps or dist(b, c) <= eps or lac <= eps:
                continue
            h = abs(cross(a[0], a[1], c[0], c[1], b[0], b[1])) / lac
            if h <= eps:
                continue
            out.append(b)
        if len(out) == m:
            break
        pts = out
        if len(pts) < 3:
            break
    return pts

def polygon_area(poly) -> float:
    n = len(poly)
    if n < 3:
        return 0.0
    s = 0.0
    for i in range(n):
        x1, y1 = poly[i]
        x2, y2 = poly[(i + 1) % n]
        s += x1 * y2 - x2 * y1
    return abs(s) / 2.0

def _area2(poly, i: int, j: int, k: int) -> float:
    ax, ay = poly[i]
    bx, by = poly[j]
    cx, cy = poly[k]
    return abs((bx - ax) * (cy - ay) - (by - ay) * (cx - ax))

def rotating_calipers_diameter(poly):
    poly = rotate_to_ccw(list(poly))
    n = len(poly)
    if n == 0:
        return 0.0, -1, -1
    if n == 1:
        return 0.0, 0, 0
    if n == 2:
        return dist(poly[0], poly[1]), 0, 1
    best = -1.0
    bi = bj = 0
    k = 1
    advances = 0
    limit = 2 * n + 4
    for i in range(n):
        ni = (i + 1) % n
        while advances <= limit:
            nk = (k + 1) % n
```

```
            if _area2(poly, i, ni, nk) > _area2(poly, i, ni, k):
                k = nk
                advances += 1
            else:
                break
        for p, q in ((i, k), (ni, k)):
            if p == q:
                continue
            d = dist(poly[p], poly[q])
            if d > best:
                best, bi, bj = d, p, q
    return best, bi, bj

def antipodal_pairs(poly):
    poly = rotate_to_ccw(list(poly))
    n = len(poly)
    if n < 2:
        return []
    if n == 2:
        return [(0, 1)]
    tol = 1e-12 * max(1.0, _scale_of(poly)) ** 2
    pairs = set()
    k = 1
    advances = 0
    limit = 2 * n + 4
    for i in range(n):
        ni = (i + 1) % n
        while advances <= limit:
            nk = (k + 1) % n
            if _area2(poly, i, ni, nk) > _area2(poly, i, ni, k) + tol:
                k = nk
                advances += 1
            else:
                break
        a = _area2(poly, i, ni, k)
        for j in (i, ni):
            pairs.add((min(j, k), max(j, k)))
        nk = (k + 1) % n
        if abs(_area2(poly, i, ni, nk) - a) <= tol:
            for j in (i, ni):
                pairs.add((min(j, nk), max(j, nk)))
    return sorted(pairs)

def brute_force_diameter(poly):
    n = len(poly)
    if n == 0:
        return 0.0, -1, -1
    if n == 1:
```

```
        return 0.0, 0, 0
    best = -1.0
    bi = bj = 0
    for i in range(n):
        for j in range(i + 1, n):
            d = dist(poly[i], poly[j])
            if d > best:
                best, bi, bj = d, i, j
    return best, bi, bj
```

```
from __future__ import annotations
import argparse
import json
import math
import os
import sys
import localization_region as lr
import rotating_calipers as rc
HERE = os.path.dirname(os.path.abspath(__file__))
OUTDIR = os.path.join(HERE, "results")
ARENA_RADIUS = 1800.0

def solve(detections, alpha_deg: float = 1.0, name: str = "case", **kw):
    status, poly, notes = lr.region_from_bearings(detections, alpha_deg, **kw)
    res = dict(
        name=name,
        status=status,
        alpha_deg=alpha_deg,
        detections=[list(d) for d in detections],
        notes=notes,
        region=[list(p) for p in poly],
        n_vertices=len(poly),
        area=rc.polygon_area(poly) if poly else 0.0,
        diameter=0.0,
        diameter_pair=None,
        diameter_pair_brute=None,
        calipers_vs_brute=0.0,
        antipodal_pairs=[],
        n_antipodal=0,
    )
    if status != "ok" or len(poly) < 2:
        return res
    d_cal, i_cal, j_cal = rc.rotating_calipers_diameter(poly)
    d_bf, i_bf, j_bf = rc.brute_force_diameter(poly)
    ap = rc.antipodal_pairs(poly)
    res.update(
        diameter=d_bf,
```

```
        diameter_pair=[int(i_cal), int(j_cal)],
        diameter_pair_brute=[int(i_bf), int(j_bf)],
        calipers_vs_brute=abs(d_cal - d_bf),
        antipodal_pairs=[[int(a), int(b)] for a, b in ap],
        n_antipodal=len(ap),
    )
    return res

def report(res) -> None:
    print("=" * 76)
    print(f"算例: {res['name']}")
    print("=" * 76)
    print(f"检测点数 k = {len(res['detections'])}, 角楔半角  = {res['alpha_deg']:g}°")
    for i, (x, y, th) in enumerate(res["detections"], 1):
        print(f"    S{i}: ({x:12.4f}, {y:12.4f})   示向度 = {th:.4f}°")
    for n in res["notes"]:
        print(f"  [约束] {n}")
    if res["status"] != "ok":
        print(f"\n区域状态: {res['status']} —— 无法给出有限直径")
        return
    print(f"\n定位区域: {res['n_vertices']} 顶点, 面积 {res['area']:.4f} m²")
    for i, (x, y) in enumerate(res["region"]):
        print(f"    V{i}: ({x:14.6f}, {y:14.6f})")
    i, j = res["diameter_pair"]
    print(f"\n【旋转卡壳】直径 D = {res['diameter']:.9f} m, 端点 V{i} ---- V{j}")
    print(f"    枚举到的对踵点对 {res['n_antipodal']} 个"
          f"(上界 3n = {3 * res['n_vertices']}), "
          f"顶点对总数 C(n,2) = {res['n_vertices'] * (res['n_vertices'] - 1) // 2}")
    print(f"    对踵点对: {res['antipodal_pairs']}")
    print(f"【交叉校验】暴力枚举 O(n²) 给出 D = {res['diameter']:.9f} m, "
          f"端点 {res['diameter_pair_brute']}, "
          f"两种方法偏差 = {res['calipers_vs_brute']:.3e} m")

def _bearings_from_source(G, pts):
    return [
        math.degrees(math.atan2(G[1] - py, G[0] - px)) % 360.0 for px, py in pts
    ]

def demo_cases():
    cases = {}
    def make(key, name, G, pts, **kw):
        bs = _bearings_from_source(G, pts)
        cases[key] = dict(
            name=name,
            alpha_deg=kw.pop("alpha_deg", 1.0),
            detections=[[px, py, b] for (px, py), b in zip(pts, bs)],
            true_source=list(G),
            **kw,
```

```
        )
    G = (150.0, -250.0)
    make("case1", "case1 双站正交交会（交会角 90°）", G,
         [(G[0] + 1200.0, G[1]), (G[0], G[1] + 1200.0)])
    make("case2", "case2 三站交会", G,
         [(G[0] + 1200.0, G[1]),
          (G[0], G[1] + 1200.0),
          (G[0] + 1300.0 * math.cos(math.radians(210.0)),
           G[1] + 1300.0 * math.sin(math.radians(210.0)))])
    make("case3", "case3 双站小交会角 5°（细长区域）", G,
         [(G[0] + 1200.0, G[1]),
          (G[0] + 1300.0 * math.cos(math.radians(5.0)),
           G[1] + 1300.0 * math.sin(math.radians(5.0)))])
    make("case4", "case4 双站同方位（方向锥重叠 → 区域无界）", G,
         [(G[0] + 1200.0, G[1]), (G[0] + 1300.0, G[1])])
    cases["case4b"] = dict(cases["case4"])
    cases["case4b"]["name"] = "case4b 同方位 + 1800 m 圆域约束（变为有界）"
    cases["case4b"]["region_disk_radius"] = ARENA_RADIUS
    make("case5", "case5 四站交会", G,
         [(G[0] + 1200.0, G[1]),
          (G[0], G[1] + 1200.0),
          (G[0] + 1300.0 * math.cos(math.radians(210.0)),
           G[1] + 1300.0 * math.sin(math.radians(210.0))),
          (G[0] + 1400.0 * math.cos(math.radians(300.0)),
           G[1] + 1400.0 * math.sin(math.radians(300.0)))])
    multi = [
        (20.0, 1200.0), (120.0, 1350.0), (225.0, 1100.0), (310.0, 1450.0),
        (55.0, 1250.0), (140.0, 1300.0), (275.0, 1150.0), (35.0, 1400.0),
        (190.0, 1050.0), (105.0, 1420.0), (290.0, 1180.0), (205.0, 1330.0),
    ]
    def multi_case(key, name, k):
        pts = []
        for ang, rr in multi[:k]:
            a = math.radians(ang)
            pts.append((G[0] + rr * math.cos(a), G[1] + rr * math.sin(a)))
        bs = _bearings_from_source(G, pts)
        cases[key] = dict(
            name=name,
            alpha_deg=1.0,
            detections=[[px, py, b] for (px, py), b in zip(pts, bs)],
            true_source=list(G),
        )
    for k, key in ((6, "case6"), (8, "case7"), (12, "case8")):
        multi_case(key, f"{key} {k}站交会", k)
    return cases

def run_case(case, save=True):
    dets = [tuple(d) for d in case["detections"]]
```

```
    kw = {}
    for k in ("region_disk_radius", "reception_max"):
        if case.get(k):
            kw[k] = case[k]
    res = solve(dets, case.get("alpha_deg", 1.0), name=case["name"], **kw)
    report(res)
    if save:
        os.makedirs(OUTDIR, exist_ok=True)
        key = case.get("key", "case")
        with open(os.path.join(OUTDIR, f"{key}.json"), "w", encoding="utf-8") as fh:
            json.dump(res, fh, ensure_ascii=False, indent=2)
    print()
    return res

def main(argv=None) -> int:
    ap = argparse.ArgumentParser(description="问题 1 前半问：旋转卡壳求定位区域直径")
    ap.add_argument("--demo", default=None)
    ap.add_argument("--list", action="store_true")
    ap.add_argument("--json", default=None)
    args = ap.parse_args(argv)
    cases = demo_cases()
    if args.list:
        for k, v in cases.items():
            print(f"{k}\t{v['name']}")
        return 0
    if args.json:
        with open(args.json, encoding="utf-8") as fh:
            case = json.load(fh)
        case.setdefault("key", os.path.splitext(os.path.basename(args.json))[0])
        run_case(case)
        return 0
    if args.demo:
        if args.demo == "all":
            sel = list(cases)
        elif args.demo in cases:
            sel = [args.demo]
        else:
            sel = [k for k in cases if args.demo in k]
        if not sel:
            print(f"未找到算例 {args.demo!r}")
            return 2
        for k in sel:
            c = dict(cases[k])
            c["key"] = k
            run_case(c)
        return 0
    ap.print_help()
    return 0
```

```

if __name__ == "__main__":
    try:
        sys.stdout.reconfigure(encoding="utf-8")
    except Exception:
        pass
    raise SystemExit(main())
```

B 问题二求解代码

```
from __future__ import annotations
import math
import os
import sys
import numpy as np
HERE = os.path.dirname(os.path.abspath(__file__))
P1 = os.path.join(os.path.dirname(HERE), "problem1(part1)")
if os.path.isdir(P1):
    sys.path.insert(0, P1)
ARENA = 1800.0
R_MIN = 1000.0
T_RECV = 1500.0
T_NEAR = 5.0
ALPHA = 1.0
def unit(deg):
    r = math.radians(deg)
    return np.array([math.cos(r), math.sin(r)])

def _wedge_hps_arr(sx, sy, th_deg, alpha=ALPHA):
    """一个角楔的 3 个半平面，返回 (3,3) 数组，每行 (a,b,c) 表示 a x + b y <= c。"""
    th, al = math.radians(th_deg), math.radians(alpha)
    ux, uy = math.cos(th), math.sin(th)
    t = math.tan(al)
    cuS = ux * sy - uy * sx
    duS = ux * sx + uy * sy
    return np.array([
        [-uy - t * ux, ux - t * uy, cuS - t * duS],
        [uy - t * ux, -ux - t * uy, -cuS - t * duS],
        [-ux, -uy, -duS],
    ])

def region_diameter(S1, th1, S2, th2, tol=1e-6):
    H = np.vstack([_wedge_hps_arr(S1[0], S1[1], th1),
                   _wedge_hps_arr(S2[0], S2[1], th2)])
    angs = np.sort(np.arctan2(H[:, 1], H[:, 0]))
    gaps = np.diff(np.concatenate([angs, angs[:1] + 2 * math.pi]))
```

```
    if gaps.max() >= math.pi - 1e-12:
        return None
    iu = np.triu_indices(6, 1)
    a1, b1, c1 = H[iu[0], 0], H[iu[0], 1], H[iu[0], 2]
    a2, b2, c2 = H[iu[1], 0], H[iu[1], 1], H[iu[1], 2]
    det = a1 * b2 - a2 * b1
    good = np.abs(det) > 1e-14
    safe = np.where(good, det, 1.0)
    x = np.where(good, (c1 * b2 - c2 * b1) / safe, np.nan)
    y = np.where(good, (a1 * c2 - a2 * c1) / safe, np.nan)
    feas = good.copy()
    for a, b, c in H:
        feas &= (a * x + b * y - c) <= tol
    P = np.column_stack([x[feas], y[feas]])
    if len(P) < 2:
        return 0.0
    d2 = ((P[:, None, :] - P[None, :, :]) ** 2).sum(-1)
    return float(math.sqrt(d2.max()))

def t_arena_along(S1, phi_deg, arena=ARENA):
    u = unit(phi_deg)
    b0 = float(S1 @ u)
    disc = b0 * b0 + arena * arena - float(S1 @ S1)
    return None if disc <= 0 else -b0 + math.sqrt(disc)

def source_vertices(S1, th, r_cap):
    out = []
    for s in (+1.0, -1.0):
        phi = th + s * ALPHA
        u = unit(phi)
        t_ar = t_arena_along(S1, phi)
        r_far = max(0.0, min(r_cap, t_ar if t_ar else 0.0))
        out.append(S1 + T_NEAR * u)
        if r_far > 1e-9:
            out.append(S1 + r_far * u)
    return np.array(out)


def edge_source(S1, th, r_cap, n_edge=100):
    S1 = np.asarray(S1, float)
    pts = []
    for s in (+1.0, -1.0):
        phi = th + s * ALPHA
        u = unit(phi)
        t_ar = t_arena_along(S1, phi)
        r_hi = max(0.0, min(r_cap, t_ar if t_ar else 0.0))
        if r_hi <= T_NEAR:
            continue
```

```
        for r in np.linspace(T_NEAR, r_hi, n_edge):
            pts.append(S1 + r * u)
    return np.array(pts)


def bearing_from(S, G):
    return math.degrees(math.atan2(G[1] - S[1], G[0] - S[0])) % 360.0

def solve_case(S1, th, na=130, nb=130, n_edge=100, verbose=True):
    S1 = np.asarray(S1, float)
    u1, n1 = unit(th), unit(th + 90.0)
    Vc = source_vertices(S1, th, R_MIN)
    Vo = edge_source(S1, th, T_RECV, n_edge)
    t_max = min(T_RECV, t_arena_along(S1, th) or 0.0)
    a = np.linspace(-ARENA, T_NEAR + R_MIN, na)
    b = np.linspace(-R_MIN, R_MIN, nb)
    A, B = np.meshgrid(a, b)
    S2x = S1[0] + A * u1[0] + B * n1[0]
    S2y = S1[1] + A * u1[1] + B * n1[1]
    ok = np.ones_like(A, dtype=bool)
    for V in Vc:
        ok &= ((S2x - V[0]) ** 2 + (S2y - V[1]) ** 2) <= R_MIN ** 2
    ok &= (S2x ** 2 + S2y ** 2) <= ARENA ** 2
    D = np.full(A.shape, np.nan)
    idx = np.argwhere(ok)
    nbad = 0
    for i, j in idx:
        S2 = np.array([S2x[i, j], S2y[i, j]])
        worst = 0.0
        bad = False
        for V in Vo:
            th1v = bearing_from(S1, V)
            th2v = bearing_from(S2, V)
            d = region_diameter(S1, th1v, S2, th2v)
            if d is None:
                bad = True
                break
            worst = max(worst, d)
        if bad:
            nbad += 1
            continue
        D[i, j] = worst
    valid = ~np.isnan(D)
    Dmin = float(np.nanmin(D))
    if verbose:
        print(f"   可行点 {int(ok.sum())}, 可评估 {int(valid.sum())}, "
              f"无界/失败 {nbad}")
        print(f"   最坏定位误差最小值 D_min = {Dmin:.2f} m  "
```

```
            f"（网格最优位置 a={float(a[np.unravel_index(np.nanargmin(D), D.shape)[1]]):.0f}, "
            f"b={float(b[np.unravel_index(np.nanargmin(D), D.shape)[0]]):.0f}）")

    return dict(S1=S1, th=th, u1=u1, n1=n1, a=a, b=b, A=A, B=B,
                S2x=S2x, S2y=S2y, D=D, ok=ok, Vc=Vc, Vo=Vo,
                t_max=t_max, Dmin=Dmin,
                n_feasible=int(valid.sum()),
                cell_area=float((a[1] - a[0]) * (b[1] - b[0])))

def sub_region(r, eps):
    """D <= D_min*(1+eps) 的子区，返回布尔掩码与统计。"""
    m = (~np.isnan(r["D"])) & (r["D"] <= r["Dmin"] * (1.0 + eps))
    n = int(m.sum())
    area = n * r["cell_area"]
    return m, n, area

def main():
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    from matplotlib import font_manager
    for fam in ("Microsoft YaHei", "SimHei", "SimSun", "DejaVu Sans"):
        try:
            font_manager.findfont(fam, fallback_to_default=False)
            plt.rcParams["font.sans-serif"] = [fam]
            break
        except Exception:
            continue
    plt.rcParams["axes.unicode_minus"] = False
    CASES = [
        ("(a) 视线沿半径朝外", (0.0, 900.0), 90.0),
        ("(b) 斜穿圆域", (1200.0, 700.0), 120.0),
        ("(c) 近切向斜穿", (1500.0, 900.0), 120.0),
    ]
    EPS_LIST = [0.02, 0.05, 0.10, 0.20]
    print("=" * 104)
    print("第二问：候选区域 → 筛出定位误差最小的部分（含角楔厚度）")
    print("=" * 104)
    res = []
    for tag, S1, th in CASES:
        print(f"\n{tag}   S1={S1},  1={th}°")
        r = solve_case(S1, th)
        res.append((tag, S1, th, r))
        print(f"   阈值      子区点数     子区面积(m²)    占可行域比    该子区最坏误差")
        for eps in EPS_LIST:
            m, n, area = sub_region(r, eps)
            print(f"   {eps:>5.0%}    {n:>9}    {area:>12.0f}    "
```

```
                f"{area / (r['n_feasible'] * r['cell_area']):>10.1%}   "
                f"{r['Dmin'] * (1 + eps):>8.1f} m")
    fig, axes = plt.subplots(1, 3, figsize=(21, 7.2))
    thv = np.linspace(0, 2 * math.pi, 700)
    for ax, (tag, S1, th, r) in zip(axes, res):
        S1a = np.asarray(S1)
        ax.plot(ARENA * np.cos(thv), ARENA * np.sin(thv), color="0.45", ls="--",
                lw=1.2, zorder=5, label="目标区域 (r=1800)")
        poly = np.vstack([r["Vc"][0], r["Vc"][2], r["Vc"][3], r["Vc"][1]])
        ax.fill(poly[:, 0], poly[:, 1], color="lime", alpha=0.9, zorder=9,
                label="源可行域（含角楔厚度）")
        Dm = r["D"]
        pos = Dm[~np.isnan(Dm)]
        vmin, vmax = r["Dmin"], float(np.percentile(pos, 85))
        cs = ax.contourf(r["S2x"], r["S2y"], Dm,
                         levels=np.linspace(vmin, vmax, 25), cmap="viridis_r",
                         alpha=0.95, zorder=1, extend="max")
        m5, _, _ = sub_region(r, 0.05)
        ax.contour(r["S2x"], r["S2y"], m5.astype(float), levels=[0.5],
                   colors="red", linewidths=2.6, zorder=12)
        ax.plot([], [], color="red", lw=2.6,
                label=f"最小误差子区 $D\\leq D_{{min}}\\times1.05$")
        ax.plot([], [], color="none", label=f"$D_{{min}}$={r['Dmin']:.0f} m")
        L = max(r["t_max"] * 1.4, 300)
        ax.plot([S1a[0], S1a[0] + L * r["u1"][0]], [S1a[1], S1a[1] + L * r["u1"][1]],
                color="tab:blue", lw=1.6, alpha=0.9, zorder=10, label="视线")
        ax.plot(*S1a, "o", color="k", ms=10, zorder=13)
        ax.annotate("$S_1$", S1a, textcoords="offset points", xytext=(-26, -34),
                    fontsize=12, zorder=14)
        lim = ARENA * 1.06
        ax.set_aspect("equal")
        ax.set_xlim(-lim, lim)
        ax.set_ylim(-lim, lim)
        ax.grid(alpha=0.18)
        ax.set_xlabel("x / m")
        ax.set_ylabel("y / m")
        ax.legend(loc="lower left", fontsize=8.2)
        ax.set_title(f"{tag}\n$S_1$=({S1[0]:.0f},{S1[1]:.0f}),  1={th:.0f}°, "
                     f"$t_{{max}}$={r['t_max']:.0f} m", fontsize=11.5)
    fig.subplots_adjust(left=0.035, right=0.912, top=0.82, bottom=0.07, wspace=0.14)
    cax = fig.add_axes([0.924, 0.11, 0.011, 0.68])
    cb = fig.colorbar(cs, cax=cax)
    cb.set_label("最坏定位误差 $D$ / m（越小越好）", fontsize=11)
    fig.suptitle("第二问：把「所有能保证探测的点」筛成「定位误差最小的那部分」"
                 "（红色圈出 $D\\leq1.05D_{min}$ 的子区）", fontsize=14, y=0.965)
    p1 = os.path.join(HERE, "最小误差候选区_三场景.png")
    fig.savefig(p1, dpi=125)
    plt.close(fig)
```

```
    print("\n->", p1)
    tag, S1, th, r = res[0]
    S1a = np.asarray(S1)
    fig2, (axA, axB, axC) = plt.subplots(1, 3, figsize=(21, 6.8))
    Dm = r["D"]
    pos = Dm[~np.isnan(Dm)]
    vmin, vmax = r["Dmin"], float(np.percentile(pos, 85))
    cs = axA.contourf(r["S2x"], r["S2y"], Dm,
                      levels=np.linspace(vmin, vmax, 30), cmap="viridis_r",
                      alpha=0.95, zorder=1, extend="max")
    cols = ["red", "orange", "magenta", "cyan"]
    for eps, col in zip(EPS_LIST, cols):
        m, n, area = sub_region(r, eps)
        axA.contour(r["S2x"], r["S2y"], m.astype(float), levels=[0.5],
                    colors=col, linewidths=2.0, zorder=10)
        axA.plot([], [], color=col, lw=2.2,
                 label=f"$\\varepsilon$={eps:.0%} → {area/1e4:.0f} 万 m²")
    axA.plot(ARENA * np.cos(thv), ARENA * np.sin(thv), color="0.45", ls="--",
             lw=1.2, zorder=5)
    poly = np.vstack([r["Vc"][0], r["Vc"][2], r["Vc"][3], r["Vc"][1]])
    axA.fill(poly[:, 0], poly[:, 1], color="lime", alpha=0.85, zorder=9)
    axA.plot(*S1a, "o", color="k", ms=10, zorder=13)
    axA.annotate("$S_1$", S1a, textcoords="offset points", xytext=(-26, -34),
                 fontsize=12, zorder=14)
    fv = ~np.isnan(Dm)
    if fv.any():
        xf, yf = r["S2x"][fv], r["S2y"][fv]
        cx, cy = float(xf.mean()), float(yf.mean())
        sp = max(float(np.ptp(xf)), float(np.ptp(yf))) * 0.72 + 60
        axA.set_xlim(cx - sp, cx + sp)
        axA.set_ylim(cy - sp, cy + sp)
    axA.set_aspect("equal")
    axA.grid(alpha=0.18)
    axA.set_xlabel("x / m")
    axA.set_ylabel("y / m")
    axA.legend(loc="lower left", fontsize=8.5)
    axA.set_title("(a) 阈值越松，子区越大：四条等值线", fontsize=12)
    fig2.subplots_adjust(left=0.035, right=0.945, top=0.83, bottom=0.11, wspace=0.26)
    cax2 = fig2.add_axes([0.957, 0.15, 0.010, 0.55])
    cb2 = fig2.colorbar(cs, cax=cax2)
    cb2.set_label("最坏定位误差 $D$ / m", fontsize=10)
    eps_fine = np.linspace(0.0, 0.6, 200)
    areas, dmaxs = [], []
    for e in eps_fine:
        m, n, area = sub_region(r, float(e))
        areas.append(area / 1e4)
        dmaxs.append(r["Dmin"] * (1 + e))
    axB.plot(dmaxs, areas, color="tab:red", lw=2.6)
```

```
    for eps, col in zip(EPS_LIST, cols):
        axB.plot(r["Dmin"] * (1 + eps), sub_region(r, eps)[2] / 1e4, "o",
                 color=col, ms=12, mec="k", mew=0.8)
    axB.set_xlabel("允许的最坏定位误差 $D$ / m")
    axB.set_ylabel("子区面积 / 万 m²")
    axB.grid(alpha=0.25)
    axB.set_title("(b) 误差-面积权衡：想要更准，可选范围指数收缩", fontsize=12)
    labels = ["全部候选区"] + [f"ε={e:.0%}" for e in EPS_LIST]
    dv = [float(np.nanmax(Dm))] + [r["Dmin"] * (1 + e) for e in EPS_LIST]
    bars = axC.bar(labels, dv, color=["0.6"] + cols, alpha=0.9,
                   edgecolor="k", lw=0.8)
    for bb, vv in zip(bars, dv):
        axC.annotate(f"{vv:,.0f}", (bb.get_x() + bb.get_width() / 2, vv),
                     textcoords="offset points", xytext=(0, 5), ha="center",
                     fontsize=10)
    axC.set_yscale("log")
    axC.set_ylabel("该方案能保证的最坏定位误差 / m（对数轴）")
    axC.grid(alpha=0.25, axis="y", which="both")
    axC.tick_params(axis="x", labelrotation=20)
    axC.set_title("(c) 筛选带来的收益：最坏误差从\n"
                  "「全部候选区」的量级降到 130 m 量级", fontsize=12)

    fig2.suptitle("场景 A 的筛选细节：从「能保证探测」到「定位误差最小」",
                  fontsize=14, y=0.965)
    p2 = os.path.join(HERE, "最小误差候选区_筛选细节.png")
    fig2.savefig(p2, dpi=125)
    plt.close(fig)
    print("->", p2)
    fig3, axes3 = plt.subplots(1, 3, figsize=(21, 6.6))
    for ax, (tag, S1, th, r) in zip(axes3, res):
        S1a = np.asarray(S1)
        m5, n5, a5 = sub_region(r, 0.05)
        x = r["S2x"][m5]
        y = r["S2y"][m5]
        ax.plot(x, y, ".", color="red", ms=3)
        allp = r["S2x"][~np.isnan(r["D"])]
        allq = r["S2y"][~np.isnan(r["D"])]
        ax.plot(allp, allq, ".", color="0.75", ms=1.2,
                label=f"全部候选区（{r['n_feasible']} 点）")
        ax.plot(x, y, ".", color="red", ms=3,
                label=f"$\\varepsilon$=5% 子区（{n5} 点，{a5/1e4:.1f} 万 m²）")
        ax.plot(*S1a, "o", color="k", ms=9, zorder=10)
        ax.annotate("$S_1$", S1a, textcoords="offset points", xytext=(-24, -32),
                    fontsize=11, zorder=11)
        L = max(r["t_max"] * 1.3, 250)
        ax.plot([S1a[0], S1a[0] + L * r["u1"][0]], [S1a[1], S1a[1] + L * r["u1"][1]],
                color="tab:blue", lw=1.6, zorder=9, label="视线")
        if len(x):
```

```
            cx, cy = float(x.mean()), float(y.mean())
            sp = max(float(np.ptp(x)), float(np.ptp(y))) * 1.8 + 120
            ax.set_xlim(cx - sp, cx + sp)
            ax.set_ylim(cy - sp, cy + sp)
        ax.set_aspect("equal")
        ax.grid(alpha=0.2)
        ax.set_xlabel("x / m")
        ax.set_ylabel("y / m")
        ax.legend(loc="upper left", fontsize=8.5)
        ax.set_title(f"{tag}\n$D_{{min}}$={r['Dmin']:.0f} m, "
                     f"子区占全部的 {a5/(r['n_feasible']*r['cell_area']):.1%}",
                     fontsize=11)
    fig3.suptitle("三场景的「最小误差候选区」并列（ =5%）：形状多为一条弧带，"
                  "紧贴场地边界一侧", fontsize=14)
    fig3.tight_layout(rect=[0, 0, 1, 0.90])
    p3 = os.path.join(HERE, "最小误差候选区_三例并列.png")
    fig3.savefig(p3, dpi=125)
    plt.close(fig)
    print("->", p3)
    return [p1, p2, p3]

if __name__ == "__main__":
    try:
        sys.stdout.reconfigure(encoding="utf-8")
    except Exception:
        pass
    main()
```

C 问题三求解代码

```
from __future__ import annotations
import math
import random
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Sequence, Tuple

Point = Tuple[float, float]
ARENA_R = 1800.0
MIN_RECEIVE_R = 1000.0
MAX_RECEIVE_R = 1500.0
BEARING_ERR_DEG = 1.0
NEAR_R = 5.0
READY_RADIUS = 19.0
CIRCLE_SIDES = 180
COARSE_STEP = 125.0
FINE_STEP = 30.0
```

```
GEOM_KEEP_RATIO = 0.35
MIN_ACTIVE_BASELINE = 80.0
ADAPTIVE_COVER_TOL = 1e-6
MOVE_SPEED_MPS = 5.0
MEASURE_ACTION_S = 5.0
TIME_RESIDUAL_WEIGHT = 0.85
TIME_TO_CENTER_WEIGHT = 0.38
TIME_QUALITY_CAP_EARLY = 1.50
TIME_QUALITY_CAP_LATE = 1.18
EARLY_CENTER_PROGRESS_WEIGHT = 1.05

def sub(a: Point, b: Point) -> Point:
    return (a[0] - b[0], a[1] - b[1])

def dot(a: Point, b: Point) -> float:
    return a[0] * b[0] + a[1] * b[1]

def cross(a: Point, b: Point) -> float:
    return a[0] * b[1] - a[1] * b[0]

def dist(a: Point, b: Point) -> float:
    return math.hypot(a[0] - b[0], a[1] - b[1])

def centroid(poly: Sequence[Point]) -> Point:
    if not poly:
        return (0.0, 0.0)
    return (sum(p[0] for p in poly) / len(poly),
            sum(p[1] for p in poly) / len(poly))

def bearing_deg(a: Point, b: Point) -> float:
    return math.degrees(math.atan2(b[1] - a[1], b[0] - a[0])) % 360.0

def dedup_polygon(poly: Sequence[Point]) -> List[Point]:
    if not poly:
        return []
    out: List[Point] = []
    for p in poly:
        if not out or dist(p, out[-1]) > 1e-7:
            out.append(p)
    if len(out) >= 2 and dist(out[0], out[-1]) <= 1e-7:
        out.pop()
    return out

def clip_halfplane(poly: Sequence[Point], f) -> List[Point]:
    if not poly:
        return []
    out: List[Point] = []
    prev = poly[-1]
```

```
    fp = f(prev)
    prev_in = fp >= -1e-10
    for cur in poly:
        fc = f(cur)
        cur_in = fc >= -1e-10
        if cur_in != prev_in:
            den = fp - fc
            if abs(den) > 1e-15:
                t = fp / den
                out.append((prev[0] + t * (cur[0] - prev[0]),
                            prev[1] + t * (cur[1] - prev[1])))
        if cur_in:
            out.append(cur)
        prev, fp, prev_in = cur, fc, cur_in
    return dedup_polygon(out)

def clip_wedge(poly: Sequence[Point], station: Point,
               measured_deg: float,
               alpha_deg: float = BEARING_ERR_DEG) -> List[Point]:
    lo = math.radians(measured_deg - alpha_deg)
    hi = math.radians(measured_deg + alpha_deg)
    ulo = (math.cos(lo), math.sin(lo))
    uhi = (math.cos(hi), math.sin(hi))
    p1 = clip_halfplane(poly, lambda p: cross(ulo, sub(p, station)))
    return clip_halfplane(p1, lambda p: -cross(uhi, sub(p, station)))

def clip_disk_outer(poly: Sequence[Point], center: Point,
                    radius: float, n: int = CIRCLE_SIDES) -> List[Point]:
    out = list(poly)
    for k in range(n):
        a = 2 * math.pi * k / n
        normal = (math.cos(a), math.sin(a))
        out = clip_halfplane(
            out,
            lambda p, nrm=normal: radius - dot(nrm, sub(p, center))
        )
        if not out:
            break
    return out

def polygon_diameter(poly: Sequence[Point]) -> float:
    best = 0.0
    for i in range(len(poly)):
        for j in range(i + 1, len(poly)):
            best = max(best, dist(poly[i], poly[j]))
    return best

def sample_poly(poly: Sequence[Point], max_vertices: int = 16) -> List[Point]:
```

```
    if not poly:
        return []
    n = len(poly)
    pts = list(poly) if n <= max_vertices else [
        poly[(i * n) // max_vertices] for i in range(max_vertices)
    ]
    pts.append(centroid(poly))
    return pts

def robust_receive(point: Point, poly: Sequence[Point],
                   radius: float = MIN_RECEIVE_R) -> bool:
    return bool(poly) and max(dist(point, v) for v in poly) <= radius + 1e-7

@dataclass
class Circle:
    c: Point
    r: float

def is_in_circle(c: Circle, p: Point) -> bool:
    return dist(c.c, p) <= c.r + 1e-7

def circle_two(a: Point, b: Point) -> Circle:
    c = ((a[0] + b[0]) / 2.0, (a[1] + b[1]) / 2.0)
    return Circle(c, dist(a, b) / 2.0)

def circle_three(a: Point, b: Point, c: Point) -> Optional[Circle]:
    ax, ay = a
    bx, by = b
    cx, cy = c
    d = 2 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by))
    if abs(d) < 1e-12:
        return None
    ux = ((ax * ax + ay * ay) * (by - cy) +
          (bx * bx + by * by) * (cy - ay) +
          (cx * cx + cy * cy) * (ay - by)) / d
    uy = ((ax * ax + ay * ay) * (cx - bx) +
          (bx * bx + by * by) * (ax - cx) +
          (cx * cx + cy * cy) * (bx - ax)) / d
    cc = (ux, uy)
    return Circle(cc, dist(cc, a))

def minimum_enclosing_circle(points: Sequence[Point]) -> Circle:
    pts = list(points)
    if not pts:
        return Circle((0.0, 0.0), 0.0)
    rnd = random.Random(20260911)
    rnd.shuffle(pts)
    c: Optional[Circle] = None
```

```
    for i, p in enumerate(pts):
        if c is None or not is_in_circle(c, p):
            c = Circle(p, 0.0)
            for j in range(i):
                q = pts[j]
                if not is_in_circle(c, q):
                    c = circle_two(p, q)
                    for k in range(j):
                        r = pts[k]
                        if not is_in_circle(c, r):
                            cc = circle_three(p, q, r)
                            if cc is not None:
                                c = cc
                            else:
                                opts = [circle_two(p, q), circle_two(p, r), circle_two(q, r)]
                                opts = [o for o in opts if all(is_in_circle(o, z) for z in (p, q, r))]
                                c = min(opts, key=lambda o: o.r)
    assert c is not None
    return c

@dataclass
class Observation:
    pos: Point
    svd_deg: float

@dataclass
class SourceState:
    channel: int
    poly: List[Point]
    observations: List[Observation] = field(default_factory=list)
    cleared: bool = False

    def add_direction(self, pos: Point, svd_deg: float) -> None:
        p = clip_wedge(self.poly, pos, svd_deg)
        a = math.radians(svd_deg)
        u = (math.cos(a), math.sin(a))
        min_forward = NEAR_R * math.cos(math.radians(BEARING_ERR_DEG))
        p = clip_halfplane(p, lambda x: dot(u, sub(x, pos)) - min_forward)
        p = clip_disk_outer(p, pos, MAX_RECEIVE_R)
        if not p:
            raise RuntimeError(f"频道{self.channel}加入观测后可行域为空")
        self.poly = p
        self.observations.append(Observation(pos, svd_deg))

    @property
    def diameter(self) -> float:
        return polygon_diameter(self.poly)
```

```
208
209     @property
210     def mec(self) -> Circle:
211         return minimum_enclosing_circle(self.poly)
212
213     @property
214     def ready(self) -> bool:
215         return bool(self.poly) and self.mec.r <= READY_RADIUS
216
217 def candidate_bbox(poly: Sequence[Point], radius: float = MIN_RECEIVE_R):
218     if not poly:
219         return None
220     xmin = max(v[0] - radius for v in poly)
221     xmax = min(v[0] + radius for v in poly)
222     ymin = max(v[1] - radius for v in poly)
223     ymax = min(v[1] + radius for v in poly)
224     return None if xmin > xmax or ymin > ymax else (xmin, xmax, ymin, ymax)
225
226 def grid_points(bounds, step: float) -> List[Point]:
227     xmin, xmax, ymin, ymax = bounds
228     pts: List[Point] = []
229     nx = max(1, int(math.floor((xmax - xmin) / step)) + 1)
230     ny = max(1, int(math.floor((ymax - ymin) / step)) + 1)
231     for ix in range(nx + 1):
232         x = min(xmax, xmin + ix * step)
233         for iy in range(ny + 1):
234             y = min(ymax, ymin + iy * step)
235             pts.append((x, y))
236     pts.append(((xmin + xmax) / 2.0, (ymin + ymax) / 2.0))
237     return list(dict.fromkeys((round(x, 6), round(y, 6)) for x, y in pts))
238
239 def geometric_score(q: Point, state: SourceState) -> float:
240     s1 = state.observations[0].pos
241     worst = 0.0
242     for g in sample_poly(state.poly, 12):
243         d1 = max(dist(s1, g), 1e-6)
244         d2 = max(dist(q, g), 1e-6)
245         sin_g = abs(cross(sub(s1, g), sub(q, g))) / max(d1 * d2, 1e-12)
246         worst = max(worst, d1 * d2 / max(sin_g, 1e-4))
247     return worst
248
249 def predicted_worst_diameter(q: Point, state: SourceState) -> float:
250     worst = 0.0
251     for g in sample_poly(state.poly, 14):
252         if dist(q, g) <= NEAR_R:
253             d_here = 0.0
254         else:
255             true_b = bearing_deg(q, g)
```

```
            d_here = 0.0
            for e in (-BEARING_ERR_DEG, 0.0, BEARING_ERR_DEG):
                p = clip_wedge(state.poly, q, true_b + e)
                if p:
                    d_here = max(d_here, polygon_diameter(p))
        worst = max(worst, d_here)
    return worst

def _time_score_measure_candidate(q: Point, predicted_d: float,
                                  state: SourceState, current_pos: Point) -> float:
    move_s = dist(current_pos, q) / MOVE_SPEED_MPS
    center_leg_s = dist(q, state.mec.c) / MOVE_SPEED_MPS
    residual_s = predicted_d / MOVE_SPEED_MPS
    center_weight = (EARLY_CENTER_PROGRESS_WEIGHT
                     if len(state.observations) <= 2 else TIME_TO_CENTER_WEIGHT)
    return (move_s + MEASURE_ACTION_S
            + center_weight * center_leg_s
            + TIME_RESIDUAL_WEIGHT * residual_s)

def _filter_measure_candidates(points: Sequence[Point], state: SourceState) -> List[Point]:
    raw = [p for p in points if robust_receive(p, state.poly)]
    good = [p for p in raw
            if all(dist(p, obs.pos) >= MIN_ACTIVE_BASELINE for obs in state.observations)]
    return good or [p for p in raw if all(dist(p, obs.pos) >= 1.0 for obs in state.observations
    )]

def choose_next_measure_point(state: SourceState, current_pos: Point) -> Point:
    bounds = candidate_bbox(state.poly)
    if bounds is None:
        return state.mec.c

    coarse = _filter_measure_candidates(grid_points(bounds, COARSE_STEP), state)
    if not coarse:
        return state.mec.c

    scored = sorted((geometric_score(p, state), p) for p in coarse)
    keep_n = max(10, int(len(scored) * max(GEOM_KEEP_RATIO, 0.45)))
    keep = [p for _, p in scored[:keep_n]]
    keep.extend(sorted(coarse, key=lambda p: dist(current_pos, p))[:6])
    keep.extend(sorted(coarse, key=lambda p: dist(state.mec.c, p))[:6])
    keep = list(dict.fromkeys(keep))
    exact = []
    for p in keep:
        pd = predicted_worst_diameter(p, state)
        exact.append((_time_score_measure_candidate(p, pd, state, current_pos), pd, p))

    best_pd = min(x[1] for x in exact)
    quality_cap = TIME_QUALITY_CAP_EARLY if len(state.observations) <= 2 else
```

```
    TIME_QUALITY_CAP_LATE
    admissible = [x for x in exact if x[1] <= best_pd * quality_cap + 1e-8]
    admissible.sort(key=lambda x: (x[0], x[1]))
    return admissible[0][2]

class Problem3CoreController:
    def interleaved_search_localize_clear(self) -> None:
        p0 = (0.0, 0.0)
        self._scan_unknown_aux(p0, force=True, label="原点")
        while True:
            if len(self.sources) >= 16 and self.unknown:
                self.empty |= self.unknown
                self.unknown.clear()

            if self.unknown and self.search_scan_points:
                radius, _ = self.guaranteed_cover_radius(self.search_scan_points)
                if radius <= MIN_RECEIVE_R + ADAPTIVE_COVER_TOL:
                    self.empty |= self.unknown
                    self.unknown.clear()
            pending = [st for st in self.sources.values() if not st.cleared]
            if not self.unknown and not pending:
                break
            completion = []
            if self.unknown:
                completion, _ = self._dynamic_route_aware_completion()
            route = self._plan_region_route(completion)
            if route:
                node_kind, node_id, anchor = route[0]
                if node_kind == "cover":
                    self._scan_unknown_aux(anchor, force=True, label="动态后备覆盖点")
                    self.maybe_probe_known_at_station(anchor)
                    continue

                if node_kind == "source":
                    action = self._task_action_for_channel(node_id)
                    if action is not None:
                        self._execute_interleaved_action(action)
                        continue
            if pending:
                action = self.next_task_action()
                if action is not None:
                    self._execute_interleaved_action(action)
                    continue
            break


```

D 问题四求解代码

```python
from __future__ import annotations
import math
import random
from dataclasses import dataclass, field
from typing import List, Optional, Sequence, Tuple

Point = Tuple[float, float]
ARENA_R = 1800.0
MIN_RECEIVE_R = 1000.0
MAX_RECEIVE_R = 1500.0
BEARING_ERR_DEG = 1.0
NEAR_R = 5.0
READY_RADIUS = 19.0
CIRCLE_SIDES = 180
P4_INNER_R = 950.0
P4_OUTER_R = ARENA_R / math.cos(math.radians(15.0))
P4_RING_N = 12
P4_PROBE_FORWARD = 180.0
P4_PROBE_LATERAL = 30.0
P4_MIN_FORWARD = 40.0
P4_MIN_LATERAL = 12.0
P4_MIN_PROBE_SCALE = 1.0 / 64.0
P4_MAX_SUCCESSFUL_PROBES = 12
P4_MAX_TOTAL_PROBES_PER_SOURCE = 60
P4_ROUTE_MULTISTART = 10
P4_SPARSE_DP_MAX_N = 15

def add(a: Point, b: Point) -> Point:
    return (a[0] + b[0], a[1] + b[1])

def sub(a: Point, b: Point) -> Point:
    return (a[0] - b[0], a[1] - b[1])

def mul(a: Point, k: float) -> Point:
    return (a[0] * k, a[1] * k)

def dot(a: Point, b: Point) -> float:
    return a[0] * b[0] + a[1] * b[1]

def cross(a: Point, b: Point) -> float:
    return a[0] * b[1] - a[1] * b[0]

def dist(a: Point, b: Point) -> float:
    return math.hypot(a[0] - b[0], a[1] - b[1])

```

```
def dedup_polygon(poly: Sequence[Point]) -> List[Point]:
    if not poly:
        return []
    out: List[Point] = []
    for p in poly:
        if not out or dist(p, out[-1]) > 1e-7:
            out.append(p)
    if len(out) >= 2 and dist(out[0], out[-1]) <= 1e-7:
        out.pop()
    return out

def clip_halfplane(poly: Sequence[Point], f) -> List[Point]:
    if not poly:
        return []
    out: List[Point] = []
    prev = poly[-1]
    fp = f(prev)
    prev_in = fp >= -1e-10
    for cur in poly:
        fc = f(cur)
        cur_in = fc >= -1e-10
        if cur_in != prev_in:
            den = fp - fc
            if abs(den) > 1e-15:
                t = fp / den
                out.append((prev[0] + t * (cur[0] - prev[0]),
                            prev[1] + t * (cur[1] - prev[1])))
        if cur_in:
            out.append(cur)
        prev, fp, prev_in = cur, fc, cur_in
    return dedup_polygon(out)

def clip_wedge(poly: Sequence[Point], station: Point,
               measured_deg: float,
               alpha_deg: float = BEARING_ERR_DEG) -> List[Point]:
    lo = math.radians(measured_deg - alpha_deg)
    hi = math.radians(measured_deg + alpha_deg)
    ulo = (math.cos(lo), math.sin(lo))
    uhi = (math.cos(hi), math.sin(hi))
    p1 = clip_halfplane(poly, lambda p: cross(ulo, sub(p, station)))
    return clip_halfplane(p1, lambda p: -cross(uhi, sub(p, station)))

def clip_disk_outer(poly: Sequence[Point], center: Point,
                    radius: float, n: int = CIRCLE_SIDES) -> List[Point]:
    out = list(poly)
    for k in range(n):
        a = 2 * math.pi * k / n
        normal = (math.cos(a), math.sin(a))
```

```
        out = clip_halfplane(out, lambda p, nrm=normal: radius - dot(nrm, sub(p, center)))
        if not out:
            break
    return out

@dataclass
class Circle:
    c: Point
    r: float

def is_in_circle(c: Circle, p: Point) -> bool:
    return dist(c.c, p) <= c.r + 1e-7

def circle_two(a: Point, b: Point) -> Circle:
    c = ((a[0] + b[0]) / 2.0, (a[1] + b[1]) / 2.0)
    return Circle(c, dist(a, b) / 2.0)

def circle_three(a: Point, b: Point, c: Point) -> Optional[Circle]:
    ax, ay = a
    bx, by = b
    cx, cy = c
    d = 2 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by))
    if abs(d) < 1e-12:
        return None
    ux = ((ax * ax + ay * ay) * (by - cy) +
          (bx * bx + by * by) * (cy - ay) +
          (cx * cx + cy * cy) * (ay - by)) / d
    uy = ((ax * ax + ay * ay) * (cx - bx) +
          (bx * bx + by * by) * (ax - cx) +
          (cx * cx + cy * cy) * (bx - ax)) / d
    cc = (ux, uy)
    return Circle(cc, dist(cc, a))

def minimum_enclosing_circle(points: Sequence[Point]) -> Circle:
    pts = list(points)
    if not pts:
        return Circle((0.0, 0.0), 0.0)
    rnd = random.Random(20260911)
    rnd.shuffle(pts)
    c: Optional[Circle] = None
    for i, p in enumerate(pts):
        if c is None or not is_in_circle(c, p):
            c = Circle(p, 0.0)
            for j in range(i):
                q = pts[j]
                if not is_in_circle(c, q):
                    c = circle_two(p, q)
                    for k in range(j):
```

```
                        r = pts[k]
                        if not is_in_circle(c, r):
                            cc = circle_three(p, q, r)
                            if cc is not None:
                                c = cc
                            else:
                                opts = [circle_two(p, q), circle_two(p, r), circle_two(q, r)]
                                opts = [o for o in opts if all(is_in_circle(o, z) for z in (p, q, r))]
                                c = min(opts, key=lambda o: o.r)
    assert c is not None
    return c

@dataclass
class Observation:
    pos: Point
    svd_deg: float

@dataclass
class SourceState:
    channel: int
    poly: List[Point]
    observations: List[Observation] = field(default_factory=list)
    cleared: bool = False
    def add_direction(self, pos: Point, svd_deg: float) -> None:
        p = clip_wedge(self.poly, pos, svd_deg)
        a = math.radians(svd_deg)
        u = (math.cos(a), math.sin(a))
        min_forward = NEAR_R * math.cos(math.radians(BEARING_ERR_DEG))
        p = clip_halfplane(p, lambda x: dot(u, sub(x, pos)) - min_forward)
        p = clip_disk_outer(p, pos, MAX_RECEIVE_R)
        if not p:
            raise RuntimeError(f"频道{self.channel}加入观测后可行域为空")
        self.poly = p
        self.observations.append(Observation(pos, svd_deg))
    @property
    def mec(self) -> Circle:
        return minimum_enclosing_circle(self.poly)
    @property
    def ready(self) -> bool:
        return bool(self.poly) and self.mec.r <= READY_RADIUS

def p4_guarantee_search_points() -> List[Point]:
    pts: List[Point] = [(0.0, 0.0)]
    for k in range(P4_RING_N):
        a = math.radians(30.0 * k)
        pts.append((P4_INNER_R * math.cos(a), P4_INNER_R * math.sin(a)))
    for k in range(P4_RING_N):
```

```python
        a = math.radians(15.0 + 30.0 * k)
        pts.append((P4_OUTER_R * math.cos(a), P4_OUTER_R * math.sin(a)))
    return pts

def open_route_length(route: Sequence[Point], start: Point = (0.0, 0.0)) -> float:
    cur = start
    total = 0.0
    for p in route:
        total += dist(cur, p)
        cur = p
    return total

def nearest_neighbor_route(points: Sequence[Point], start: Point = (0.0, 0.0)) -> List[Point]:
    rem = list(points)
    route: List[Point] = []
    cur = start
    while rem:
        idx = min(range(len(rem)), key=lambda i: dist(cur, rem[i]))
        cur = rem.pop(idx)
        route.append(cur)
    return route

def two_opt_open_route(route: Sequence[Point], start: Point = (0.0, 0.0)) -> List[Point]:
    r = list(route)
    improved = True
    while improved:
        improved = False
        n = len(r)
        for i in range(n - 1):
            a = start if i == 0 else r[i - 1]
            b = r[i]
            for j in range(i + 1, n):
                c = r[j]
                dnext = None if j == n - 1 else r[j + 1]
                old = dist(a, b) + (0.0 if dnext is None else dist(c, dnext))
                new = dist(a, c) + (0.0 if dnext is None else dist(b, dnext))
                if new + 1e-9 < old:
                    r[i:j + 1] = reversed(r[i:j + 1])
                    improved = True
                    break
            if improved:
                break
    return r

def p4_search_route() -> List[Point]:
    pts = p4_guarantee_search_points()
    center = pts[0]
    return [center] + two_opt_open_route(nearest_neighbor_route(pts[1:], center), center)
```

```

class Problem4CoreController:
    def guaranteed_directional_search(self) -> None:
        center = self.search_route[0]
        self.scan_unknown_at(center, 0)
        self.opportunistic_known_at_search_station(center)
        remaining = [(i, p) for i, p in enumerate(self.search_route[1:], start=1)]
        while remaining:
            if len(self.sources) >= 16:
                self.empty |= self.unknown
                self.unknown.clear()
                break
            route = self._plan_mixed_route(remaining)
            if not route:
                break
            kind, ident, q = route[0]
            if kind == "search":
                self.scan_unknown_at(q, ident)
                self.opportunistic_known_at_search_station(q)
                remaining = [(j, p) for j, p in remaining if j != ident]
                continue
            if kind == "clear":
                self.try_clear_source(ident)
            else:
                self.try_local_clear_sweep(ident)
        if self.unknown:
            self.empty |= self.unknown
            self.unknown.clear()
        self._activate_sparse_mode()

    def _probe_offsets(self, st: SourceState, scale: float) -> Tuple[float, float]:
        obs = st.observations[-1]
        a = math.radians(obs.svd_deg)
        u = (math.cos(a), math.sin(a))
        est_forward = dot(sub(st.mec.c, obs.pos), u) if st.poly else P4_PROBE_FORWARD
        f0 = min(P4_PROBE_FORWARD, max(P4_MIN_FORWARD, 0.55 * max(est_forward, 0.0)))
        b0 = min(P4_PROBE_LATERAL, max(P4_MIN_LATERAL, 0.18 * f0))
        return f0 * scale, b0 * scale

    def _paired_probe_points(self, st: SourceState, scale: float) -> List[Point]:
        obs = st.observations[-1]
        a = math.radians(obs.svd_deg)
        u = (math.cos(a), math.sin(a))
        v = (-math.sin(a), math.cos(a))
        f, b = self._probe_offsets(st, scale)
        base = add(obs.pos, mul(u, f))
        return [add(base, mul(v, b)), add(base, mul(v, -b))]

```

```
    def _exact_open_route_dp(self, nodes, start: Point):
        n = len(nodes)
        if n <= 1:
            return list(nodes)
        full = (1 << n) - 1
        inf = float("inf")
        dp = [inf] * ((1 << n) * n)
        parent = [-1] * ((1 << n) * n)
        start_dist = [dist(start, nodes[j][2]) for j in range(n)]
        dmat = [[0.0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                d = dist(nodes[i][2], nodes[j][2])
                dmat[i][j] = dmat[j][i] = d
        for j in range(n):
            dp[(1 << j) * n + j] = start_dist[j]
        for mask in range(1, full + 1):
            for j in range(n):
                cur = dp[mask * n + j]
                if cur == inf or not (mask & (1 << j)):
                    continue
                for k in range(n):
                    if mask & (1 << k):
                        continue
                    nmask = mask | (1 << k)
                    idx = nmask * n + k
                    nd = cur + dmat[j][k]
                    if nd < dp[idx]:
                        dp[idx] = nd
                        parent[idx] = j
        end = min(range(n), key=lambda j: dp[full * n + j])
        order_rev = []
        mask = full
        while end >= 0:
            order_rev.append(end)
            prev = parent[mask * n + end]
            mask ^= 1 << end
            end = prev
        order_rev.reverse()
        return [nodes[i] for i in order_rev]
    def localize_one_source(self, channel: int) -> None:
        st = self.sources[channel]
        if st.cleared:
            return
        if st.poly and st.ready and self.try_clear_source(channel):
            return
        if st.poly and self.try_local_clear_sweep(channel):
            return
```

```
        if st.poly and self._maybe_trial_clear(channel):
            return
        successful = max(0, len(st.observations) - 1)
        total_probes = 0
        while not st.cleared:
            if total_probes >= P4_MAX_TOTAL_PROBES_PER_SOURCE:
                raise RuntimeError("定向追踪探测次数过多")

            if st.poly and st.ready and self.try_clear_source(channel):
                return
            if st.poly and self.try_local_clear_sweep(channel):
                return
            if st.poly and self._maybe_trial_clear(channel):
                return
            scale = 1.0
            got_signal = False
            while scale >= P4_MIN_PROBE_SCALE - 1e-12:
                qlist = self._paired_probe_points(st, scale)
                qlist.sort(key=lambda q: dist(self.client.current_pos, q))
                for q in qlist:
                    self._ensure_real_time()
                    total_probes += 1
                    resp = self.client.measure(q, channel)
                    result = resp.get("measure_result")
                    if result == "no_signal":
                        continue
                    self._handle_measure(channel, q, resp)
                    got_signal = True
                    if result == "near" or self.sources[channel].cleared:
                        return
                    successful += 1
                    st = self.sources[channel]
                    if st.poly and st.ready and self.try_clear_source(channel):
                        return
                    break
                if got_signal:
                    break
                # 左右均无信号：缩小探测尺度
                scale *= 0.5
            if not got_signal:
                # 极小尺度仍失败时回到最近一次成功测向点恢复
                obs = st.observations[-1]
                resp = self.client.measure(obs.pos, channel)
                if resp.get("measure_result") == "no_signal":
                    raise RuntimeError("此前成功测向点重新测量却无信号")
                self._handle_measure(channel, obs.pos, resp)

    def localize_and_clear_all(self) -> None:
```

```
        while True:
            c = self._choose_next_localize_channel()
            if c is None:
                break
            self.localize_one_source(c)
```